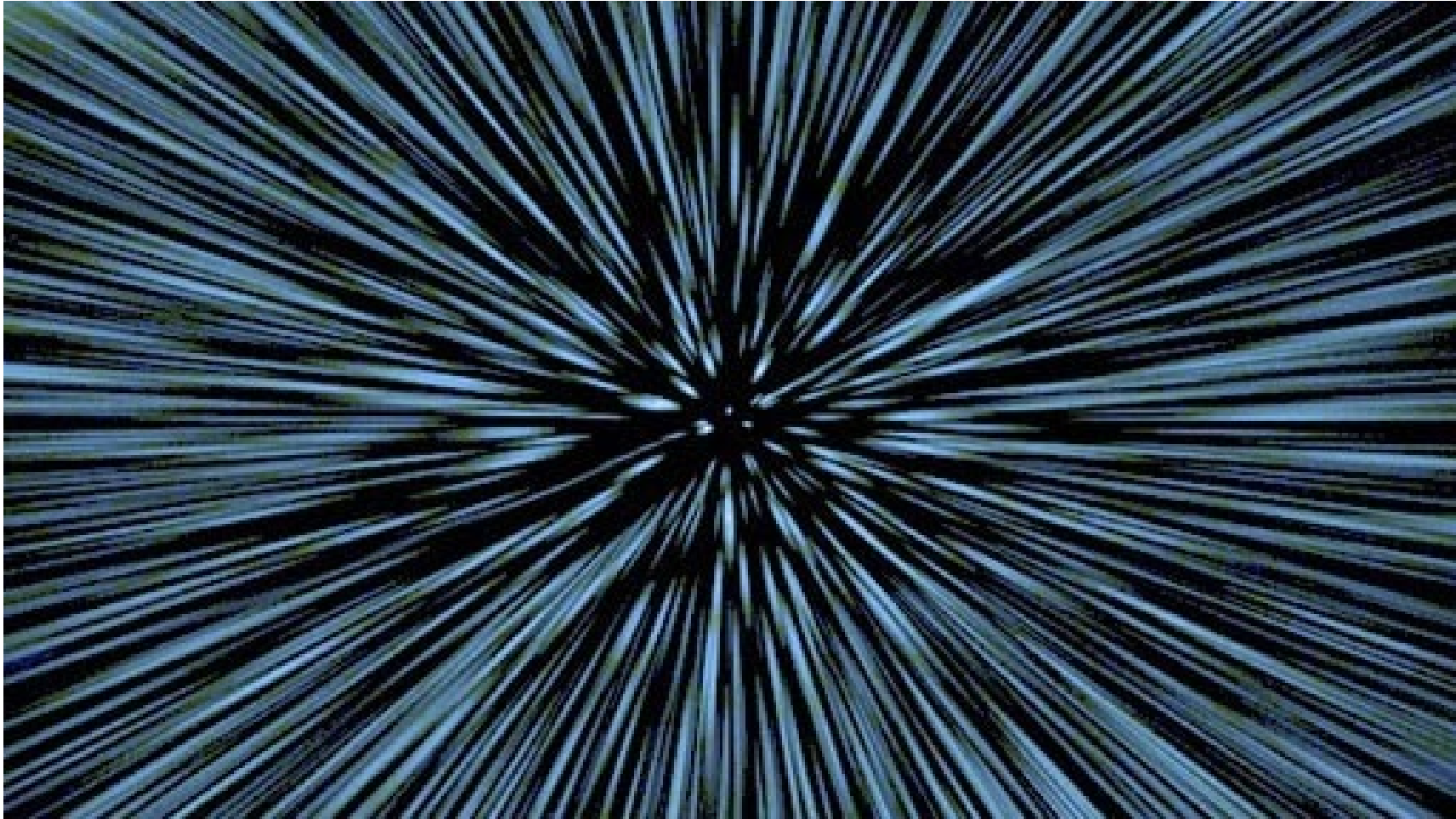


Image transformations



What is an image?

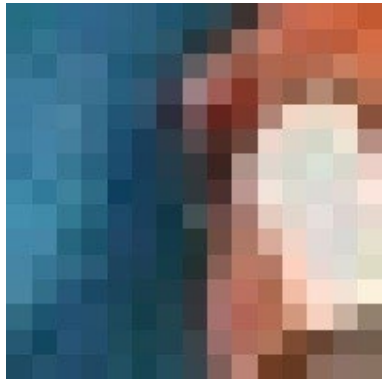


What is an image?



A (color) image
is a 3D tensor
of numbers.

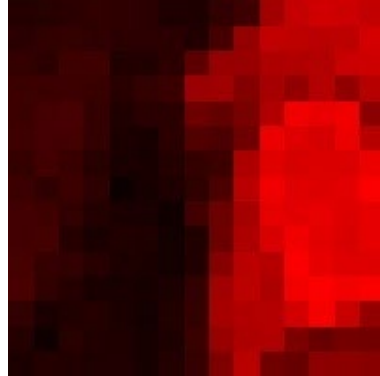
What is an image?



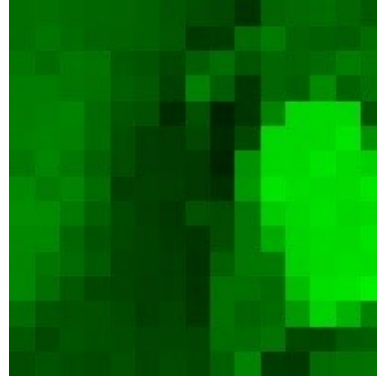
color image patch

How many bits are
the intensity values?

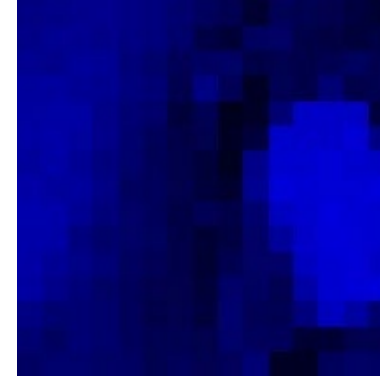
red



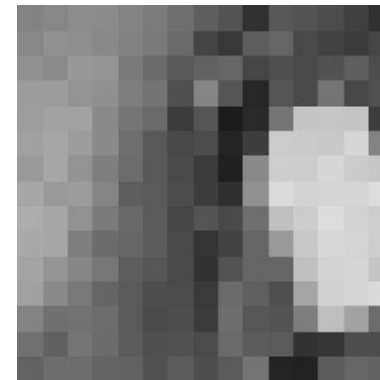
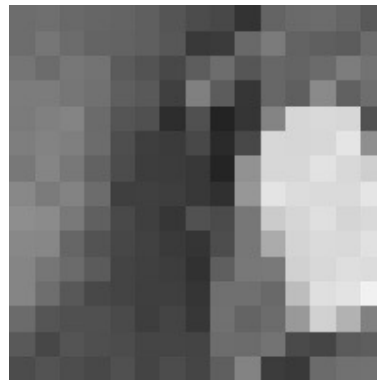
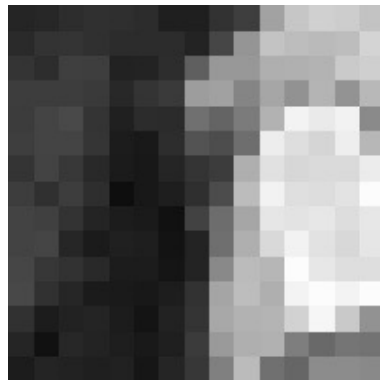
green



blue



colorized for visualization



actual intensity values per channel

Each channel
is a 2D array of
numbers.

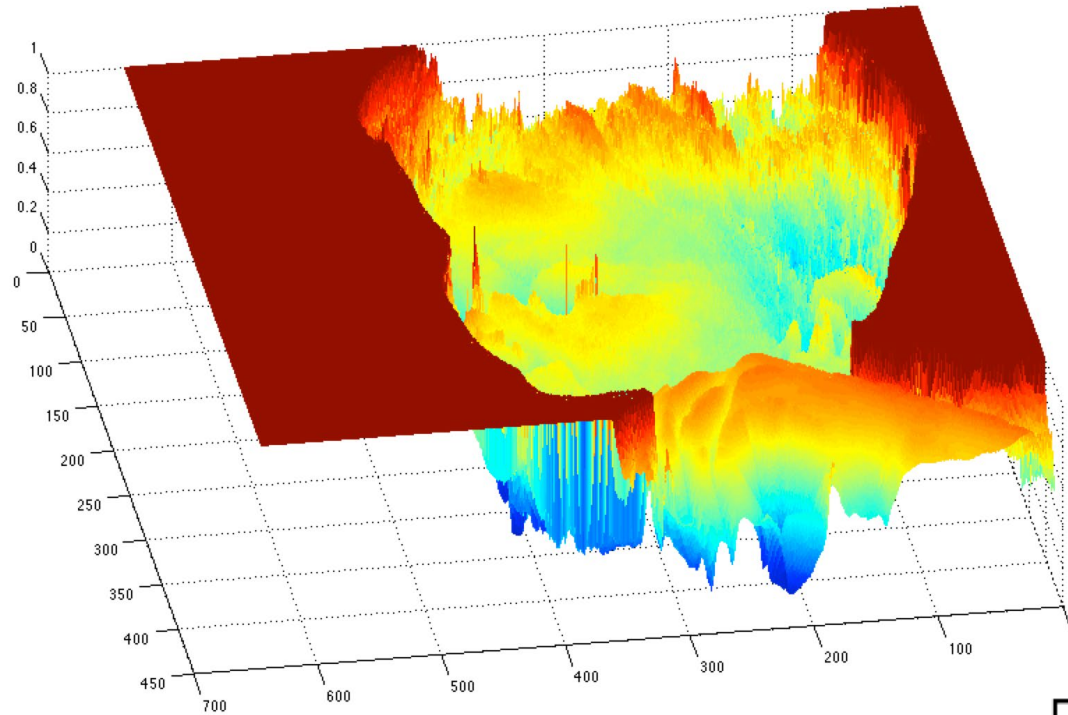
What is an image?



grayscale image

What is the range of
the image function f ?

$$f(\mathbf{x})$$



domain $\mathbf{x} = \begin{bmatrix} x \\ y \end{bmatrix}$

A (grayscale)
image is a 2D
function.

What types of image transformations can we do?



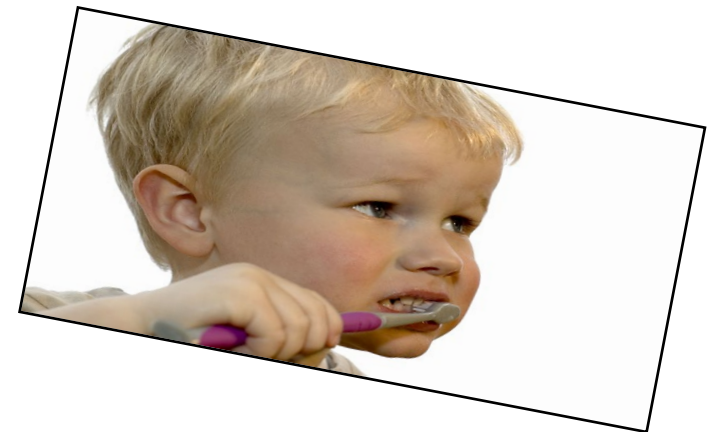
Filtering



changes pixel *values*



Warping



changes pixel *locations*

What types of image transformations can we do?

F

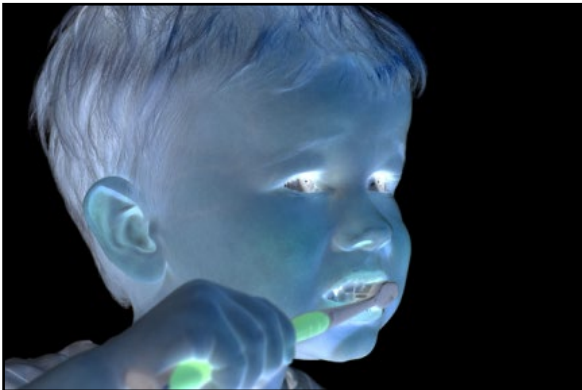


Filtering



$$G(\mathbf{x}) = h\{F(\mathbf{x})\}$$

G



changes *range* of image function

F

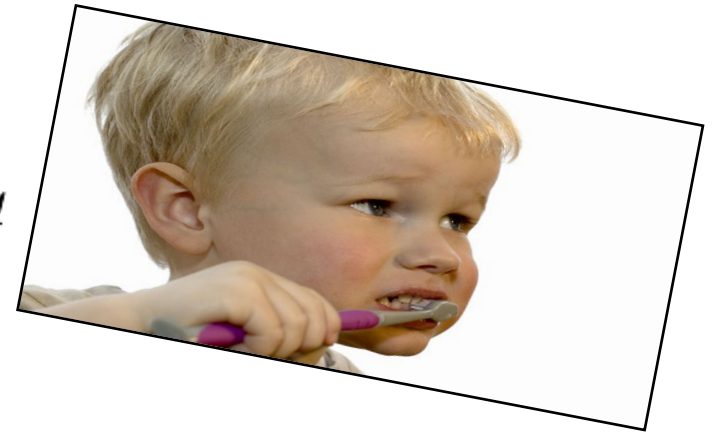


Warping



$$G(\mathbf{x}) = F(h\{\mathbf{x}\})$$

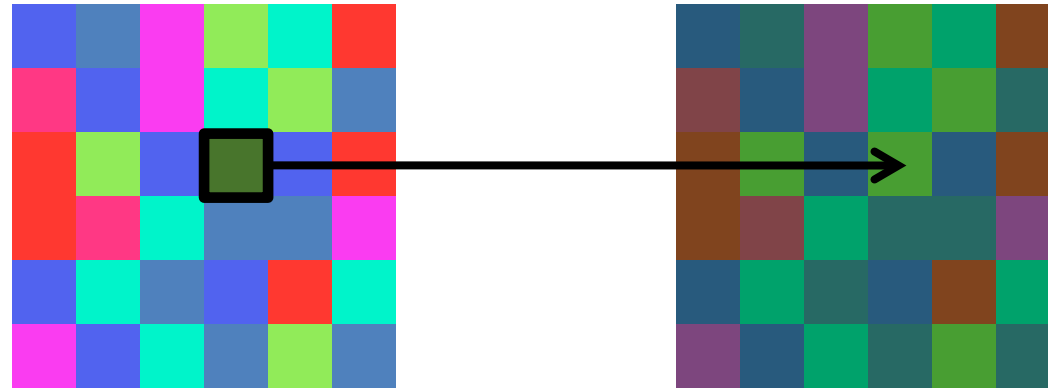
G



changes *domain* of image function

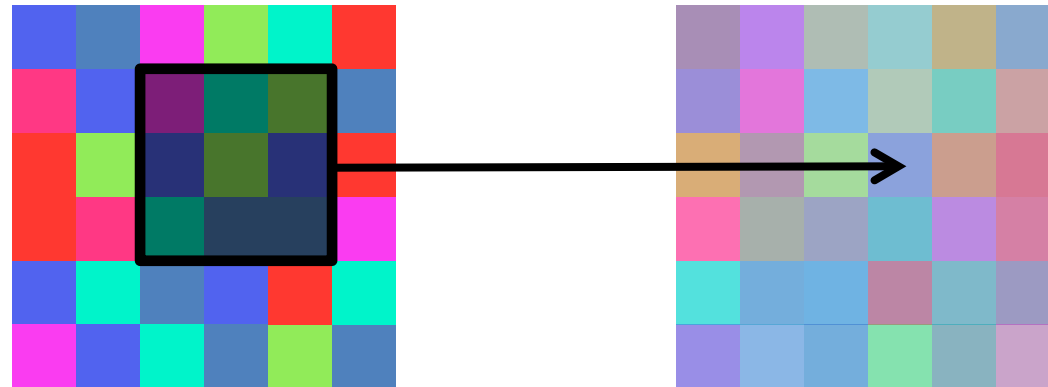
What types of image filtering can we do?

Point Operation



point processing

Neighborhood Operation



“filtering”

Point processing

Examples of point processing

original



darken



lower contrast



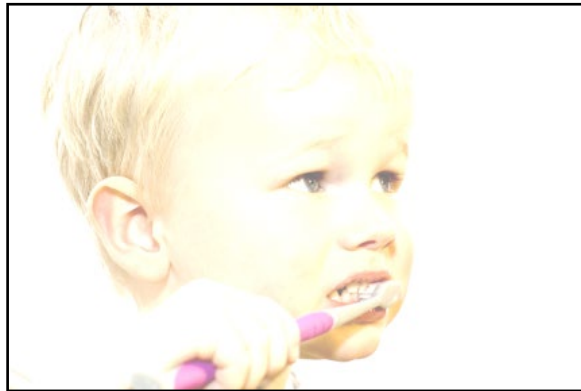
non-linear lower contrast



invert



lighten



raise contrast



non-linear raise contrast



How would you
implement these?

Examples of point processing

original

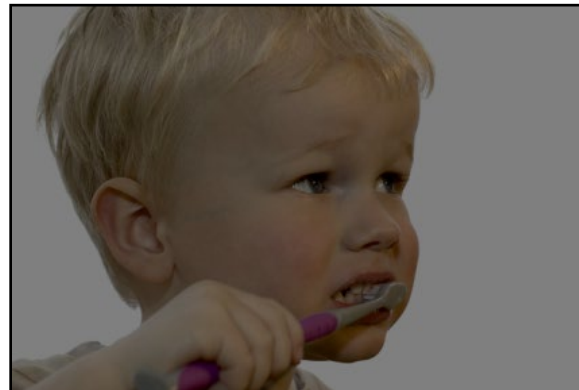


x

darken



lower contrast



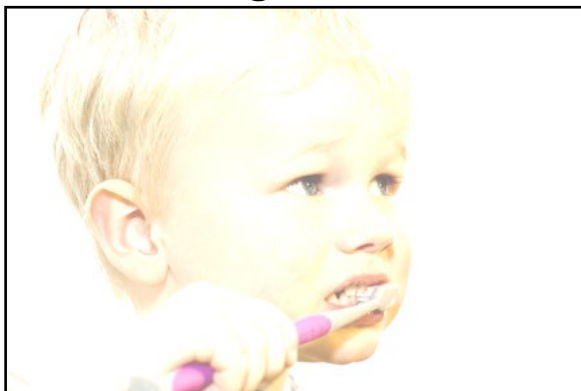
non-linear lower contrast



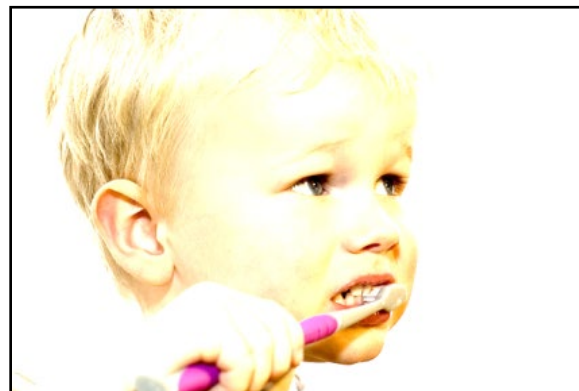
invert



lighten



raise contrast



non-linear raise contrast



How would you
implement these?

Examples of point processing

original



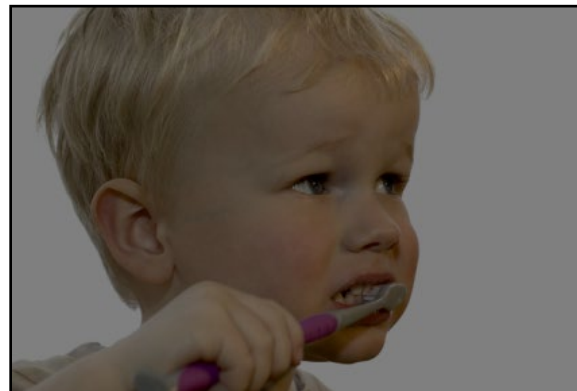
$$x$$

darken



$$x - 128$$

lower contrast



$$\frac{x}{2}$$

non-linear lower contrast



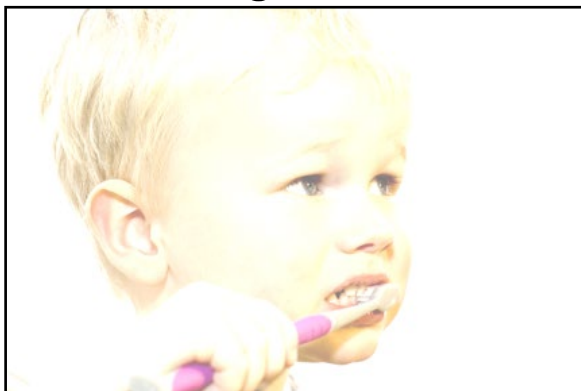
$$\left(\frac{x}{255}\right)^{1/3} \times 255$$

invert



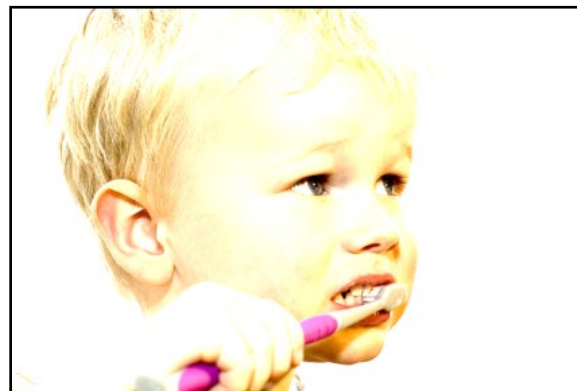
$$255 - x$$

lighten



$$x + 128$$

raise contrast



$$x \times 2$$

non-linear raise contrast



$$\left(\frac{x}{255}\right)^2 \times 255$$

Many other types of point processing



camera output



image after stylistic tonemapping

Many other types of point processing



Linear shift-invariant image filtering

Linear shift-invariant image filtering

- Replace each pixel by a *linear* combination of its neighbors (and possibly itself).
- The combination is determined by the filter's *kernel*.
- The same kernel is *shifted* to all pixel locations so that all pixels use the same linear combination of their neighbors.

Example: the box filter

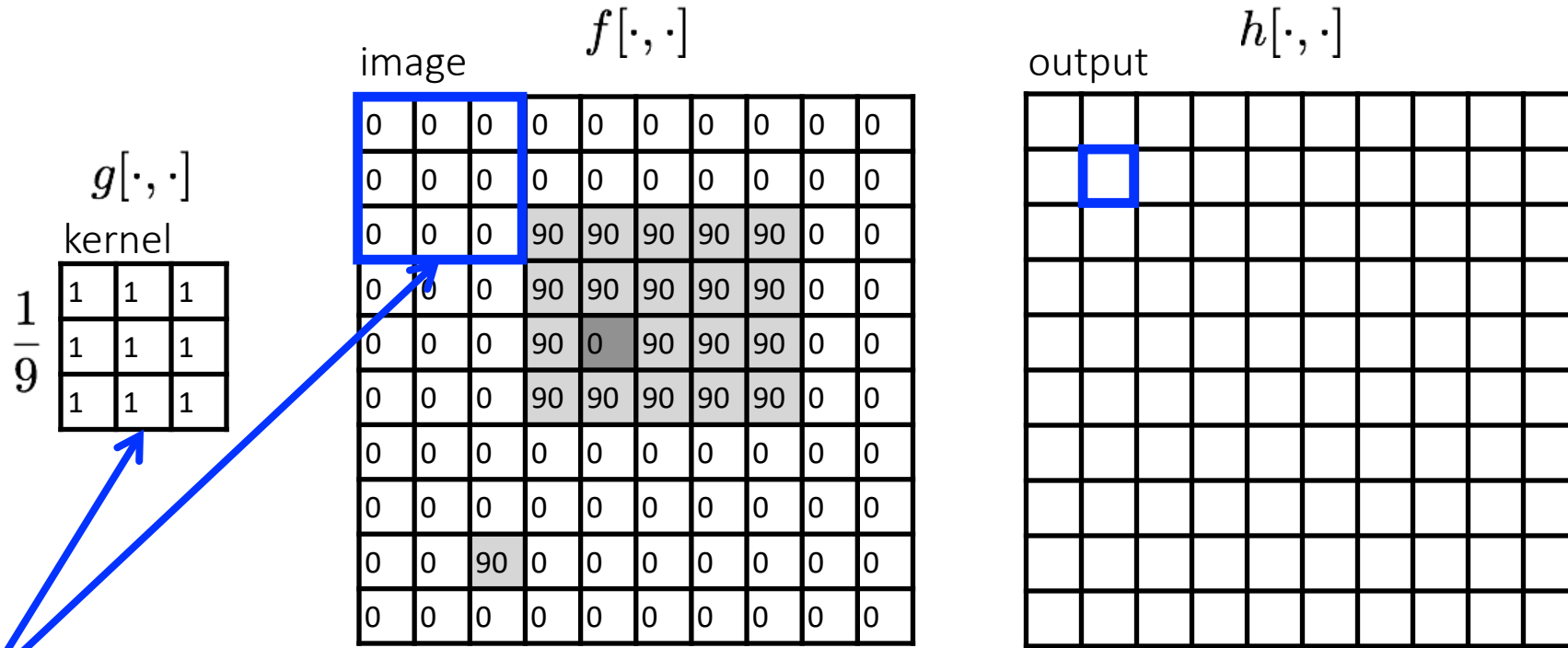
- also known as the 2D rect (not rekt) filter
- also known as the square mean filter

$$\text{kernel } g[\cdot, \cdot] = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

- replaces pixel with local average
- has smoothing (blurring) effect



Let's run the box filter

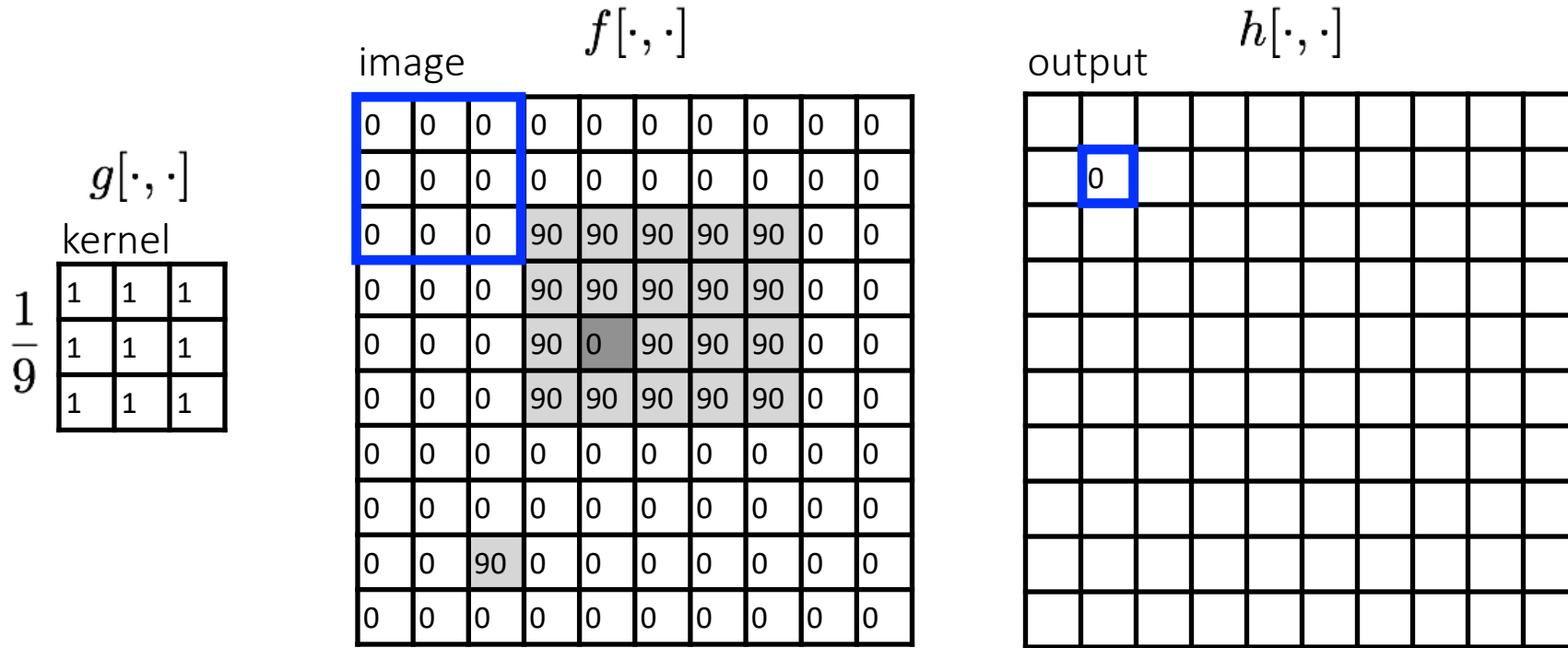


note that we assume that
the kernel coordinates are
centered

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output
 k, l
filter
image (signal)

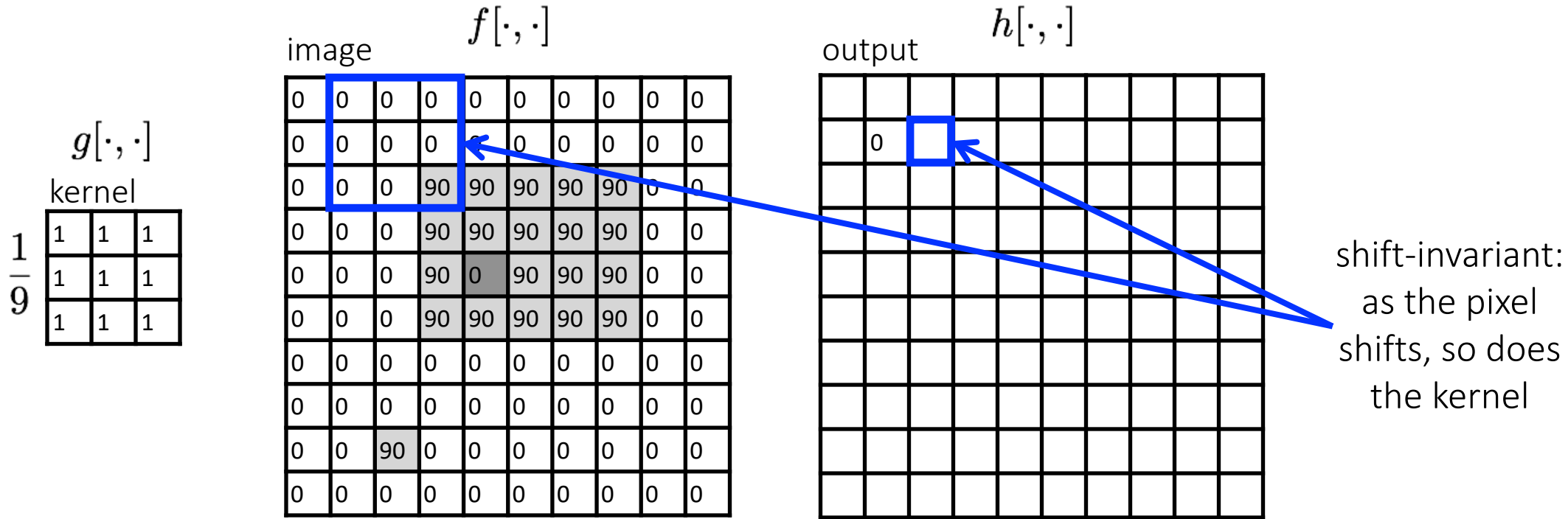
Let's run the box filter



$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output
 k, l
filter
image (signal)

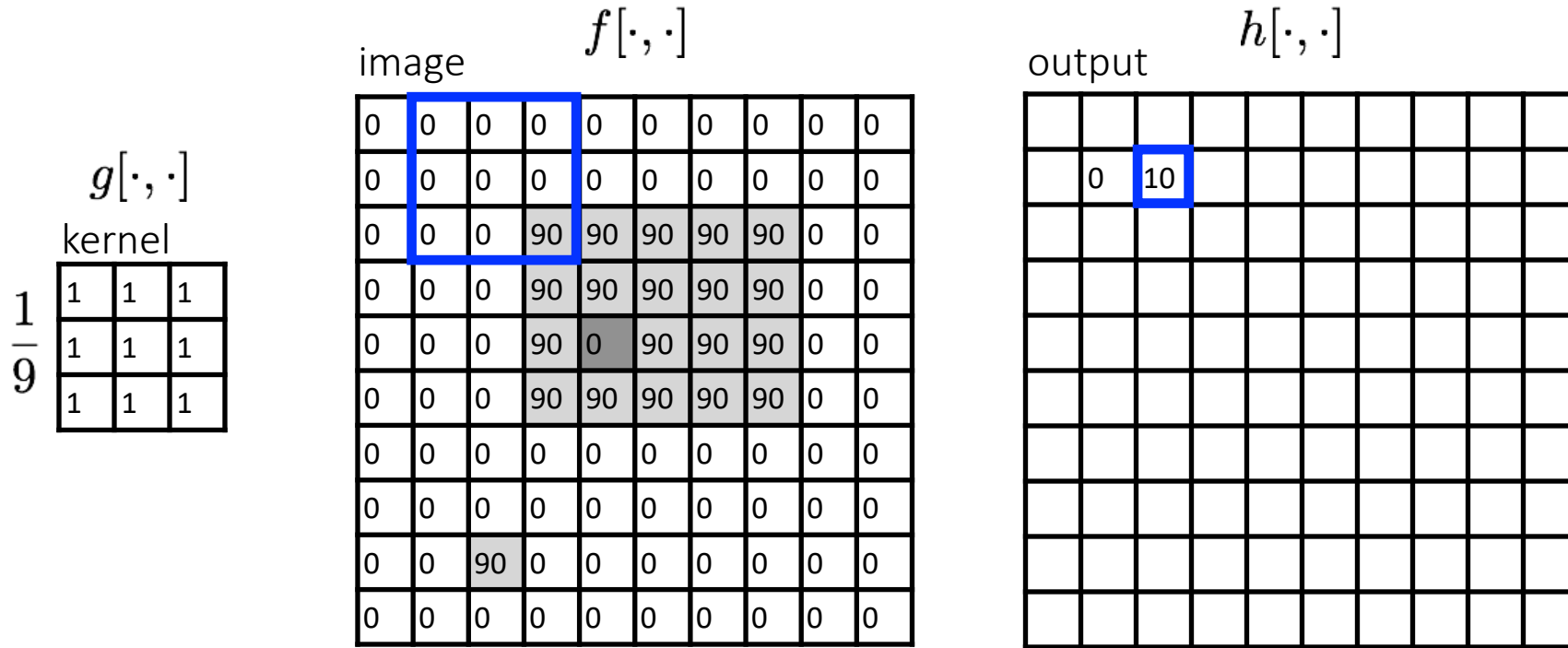
Let's run the box filter



$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output k, l filter image (signal)

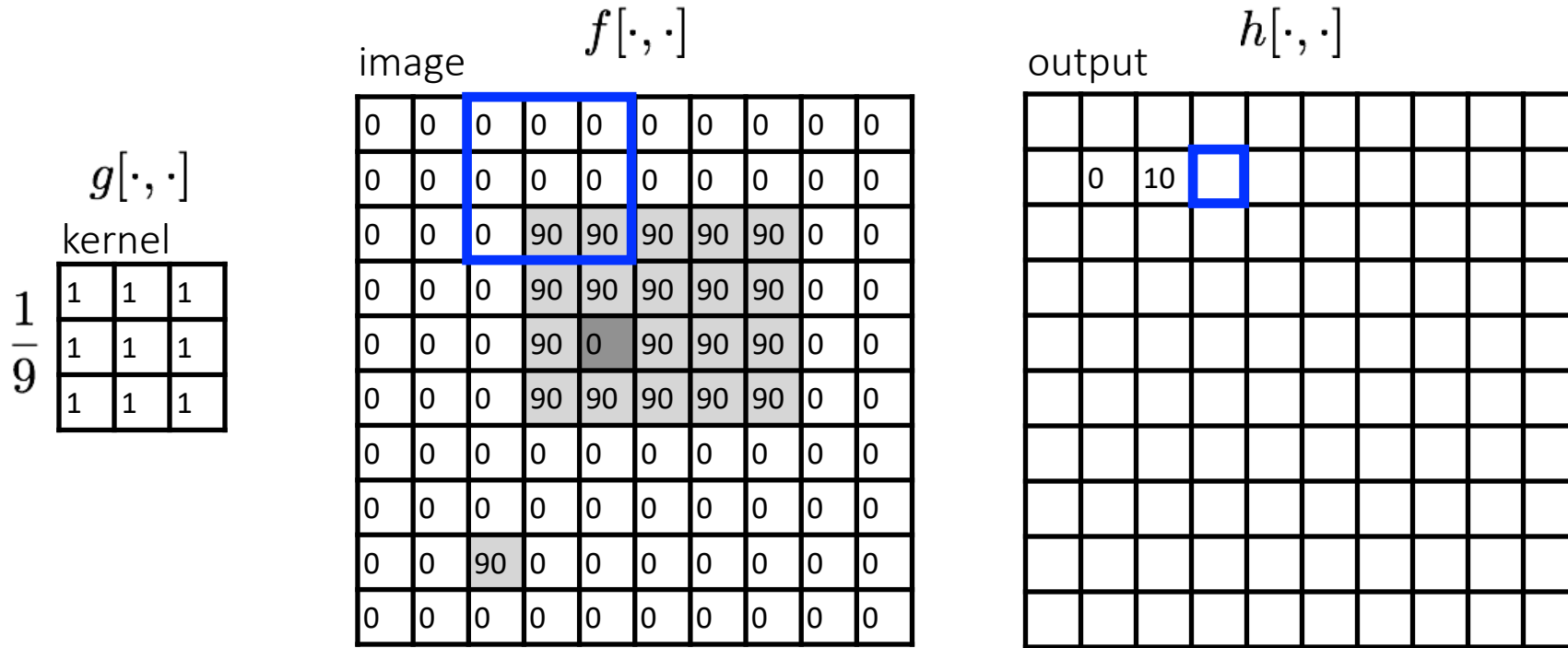
Let's run the box filter



$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output k, l filter image (signal)

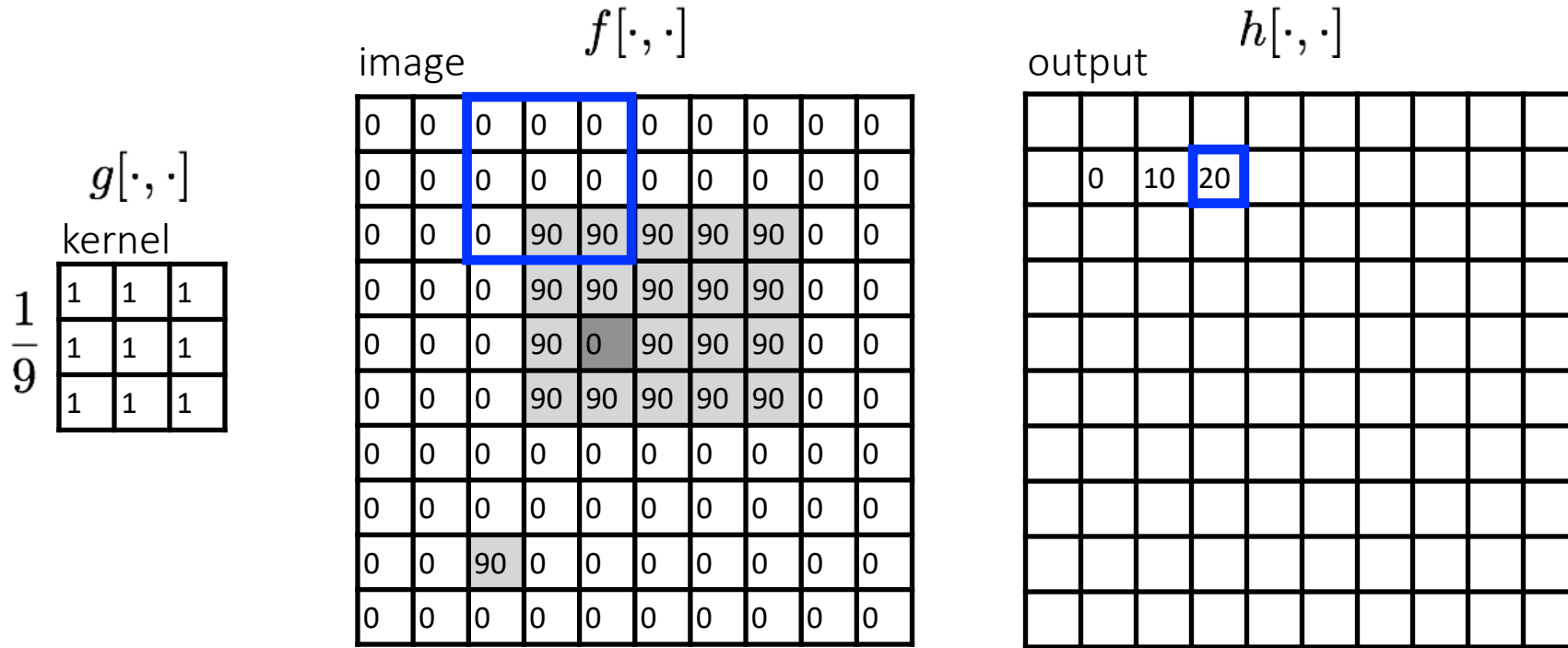
Let's run the box filter



$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output k, l filter image (signal)

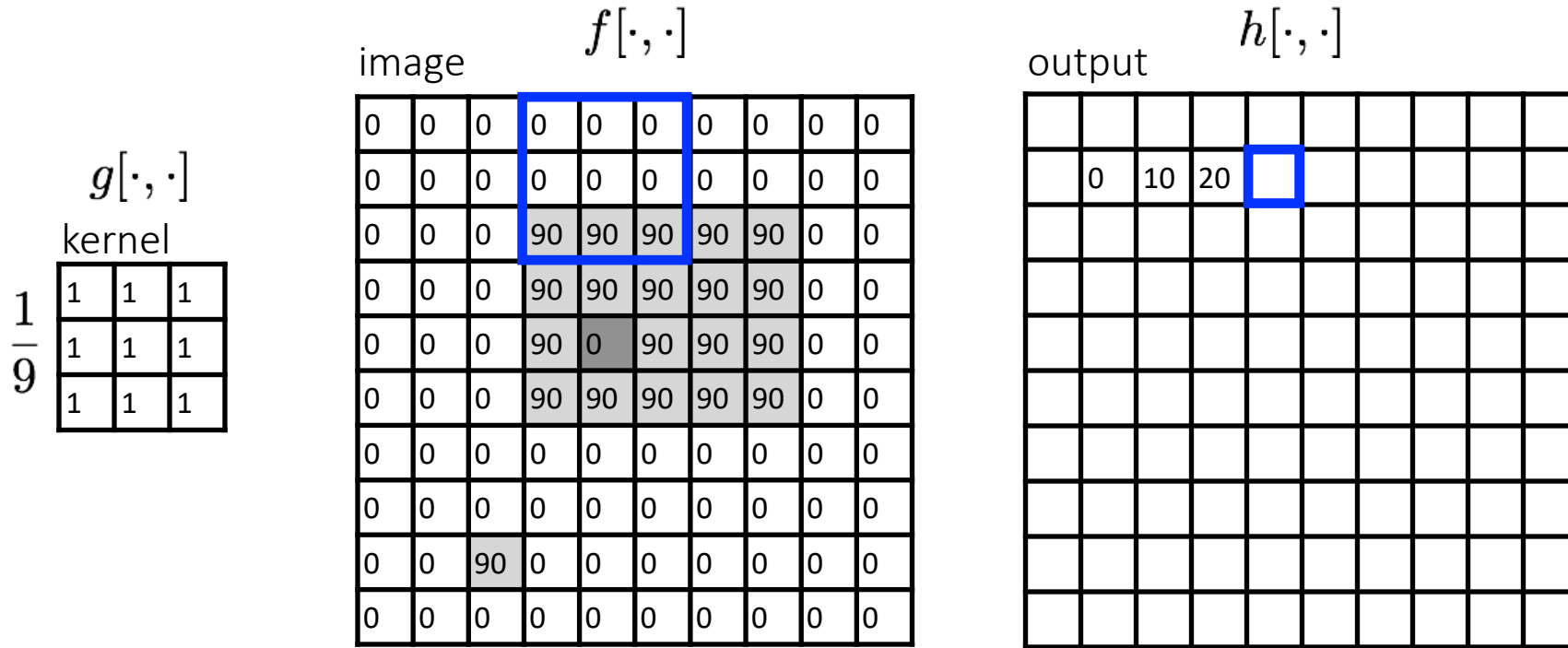
Let's run the box filter



$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output k, l filter image (signal)

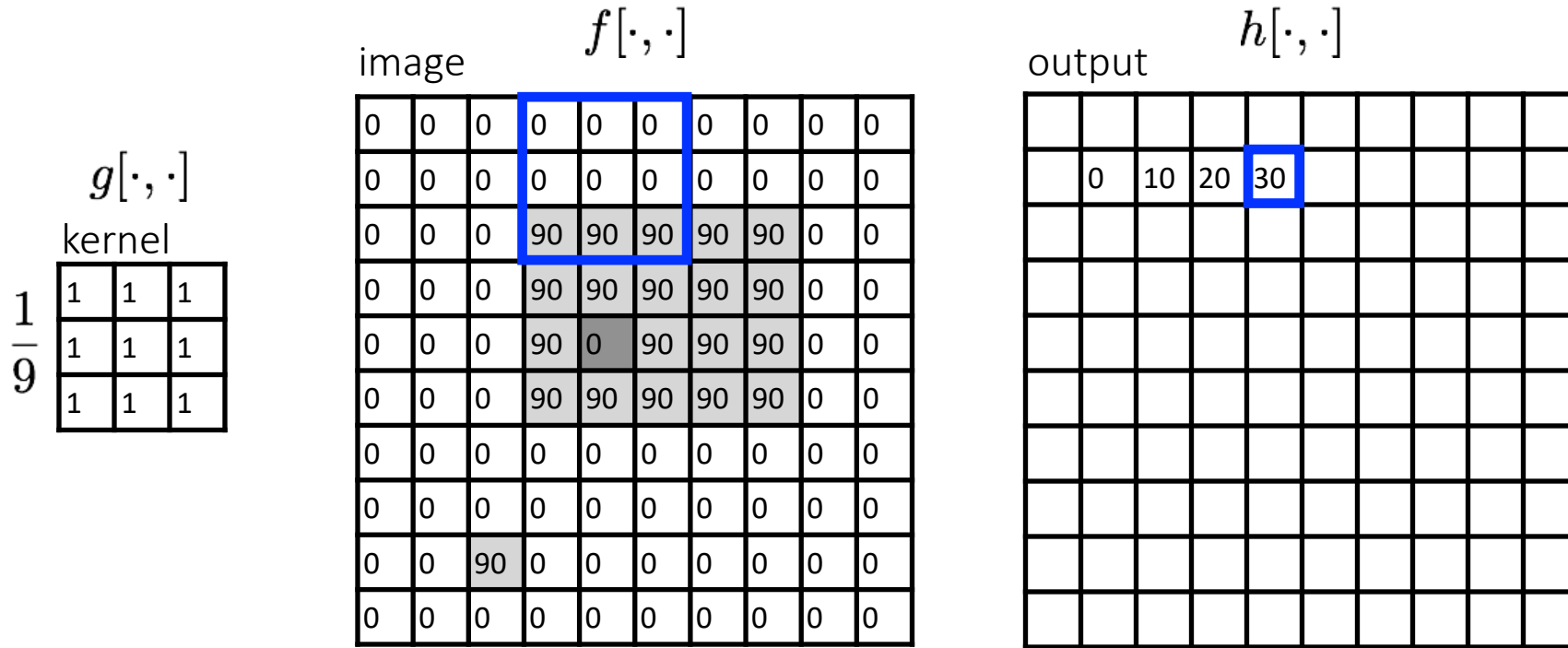
Let's run the box filter



$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output
 k, l
filter
image (signal)

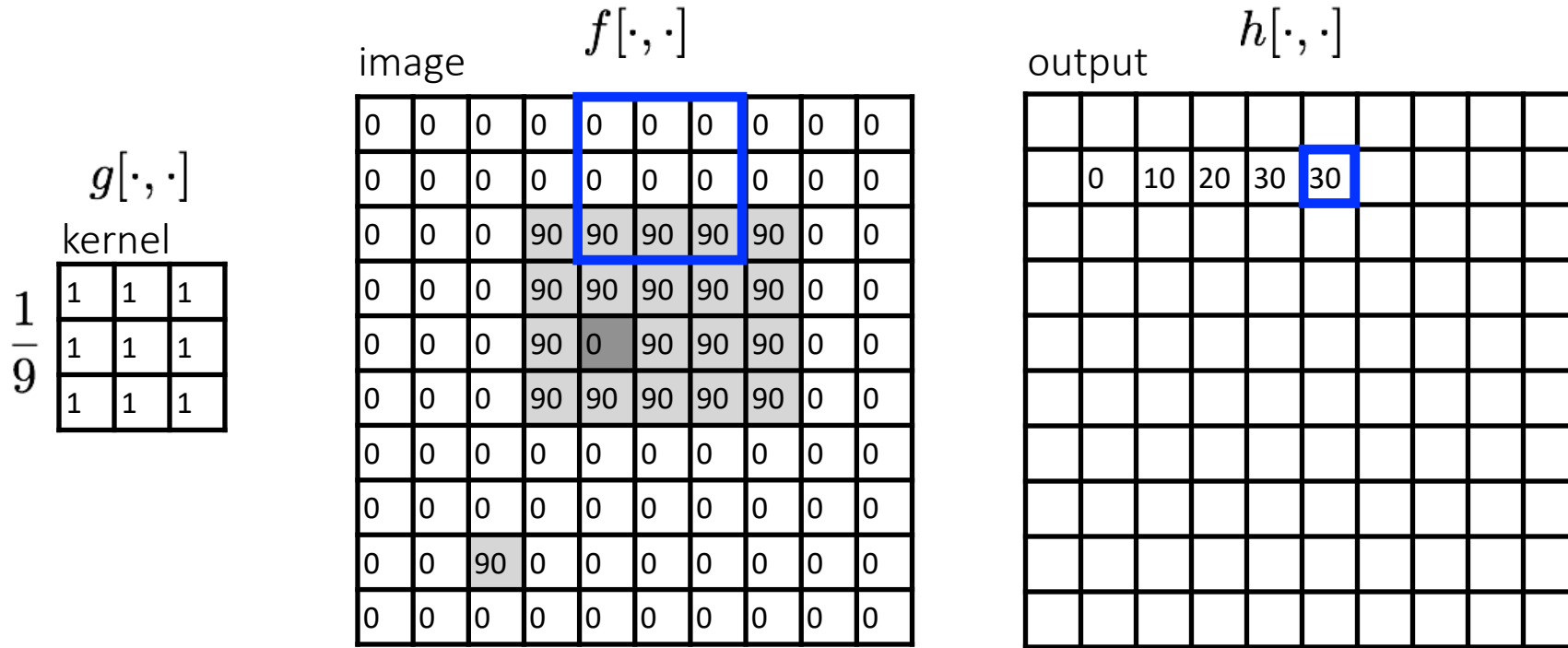
Let's run the box filter



$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output
 k, l
filter
image (signal)

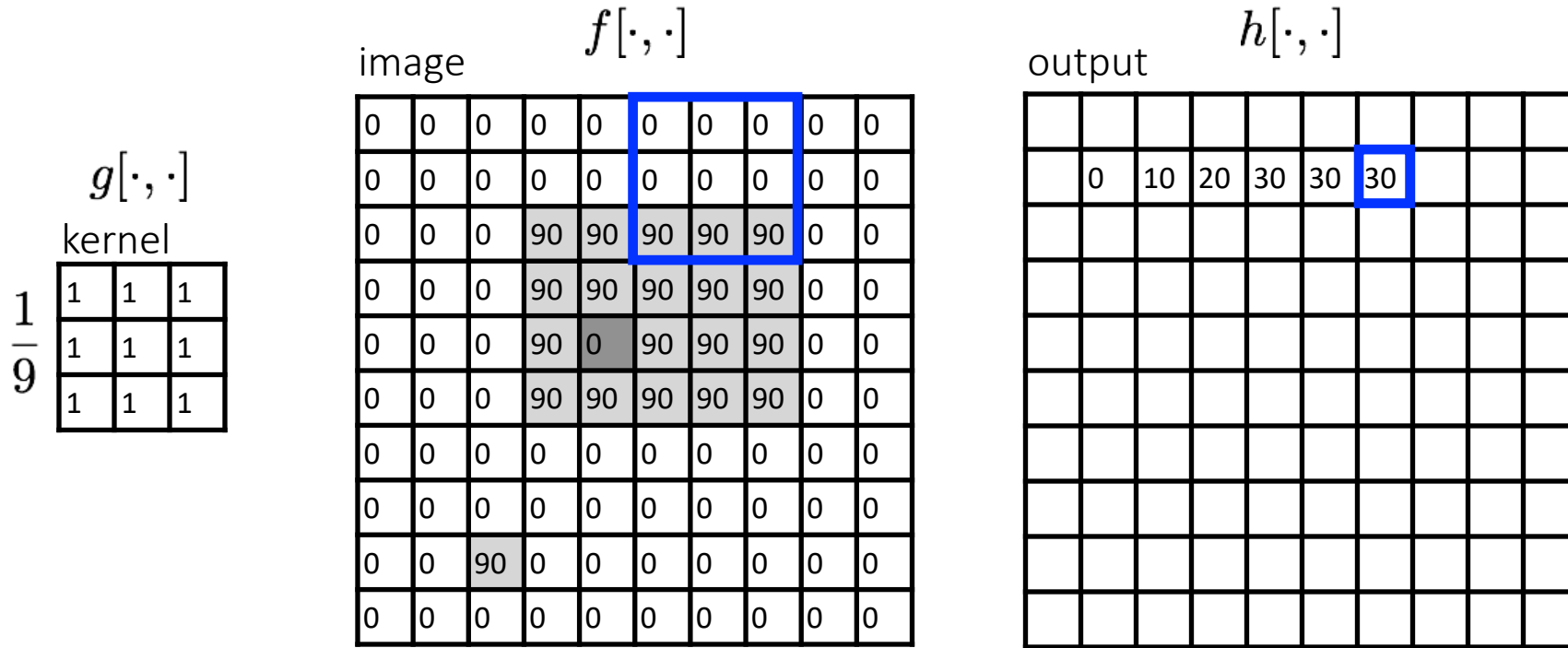
Let's run the box filter



$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output
 k, l
filter
image (signal)

Let's run the box filter



$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output
 k, l
filter
image (signal)

Let's run the box filter

$$\frac{1}{9} g[\cdot, \cdot]$$

1	1	1
1	1	1
1	1	1

kernel

image $f[\cdot, \cdot]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

output $h[\cdot, \cdot]$

	0	10	20	30	30	30	20		

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output
 k, l
filter
image (signal)

Let's run the box filter

$$\frac{1}{9} g[\cdot, \cdot]$$

1	1	1
1	1	1
1	1	1

kernel

image $f[\cdot, \cdot]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

output $h[\cdot, \cdot]$

	0	10	20	30	30	30	20	10	

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output k, l filter image (signal)

Let's run the box filter

$$\frac{1}{9} g[\cdot, \cdot]$$

1	1	1
1	1	1
1	1	1

image $f[\cdot, \cdot]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

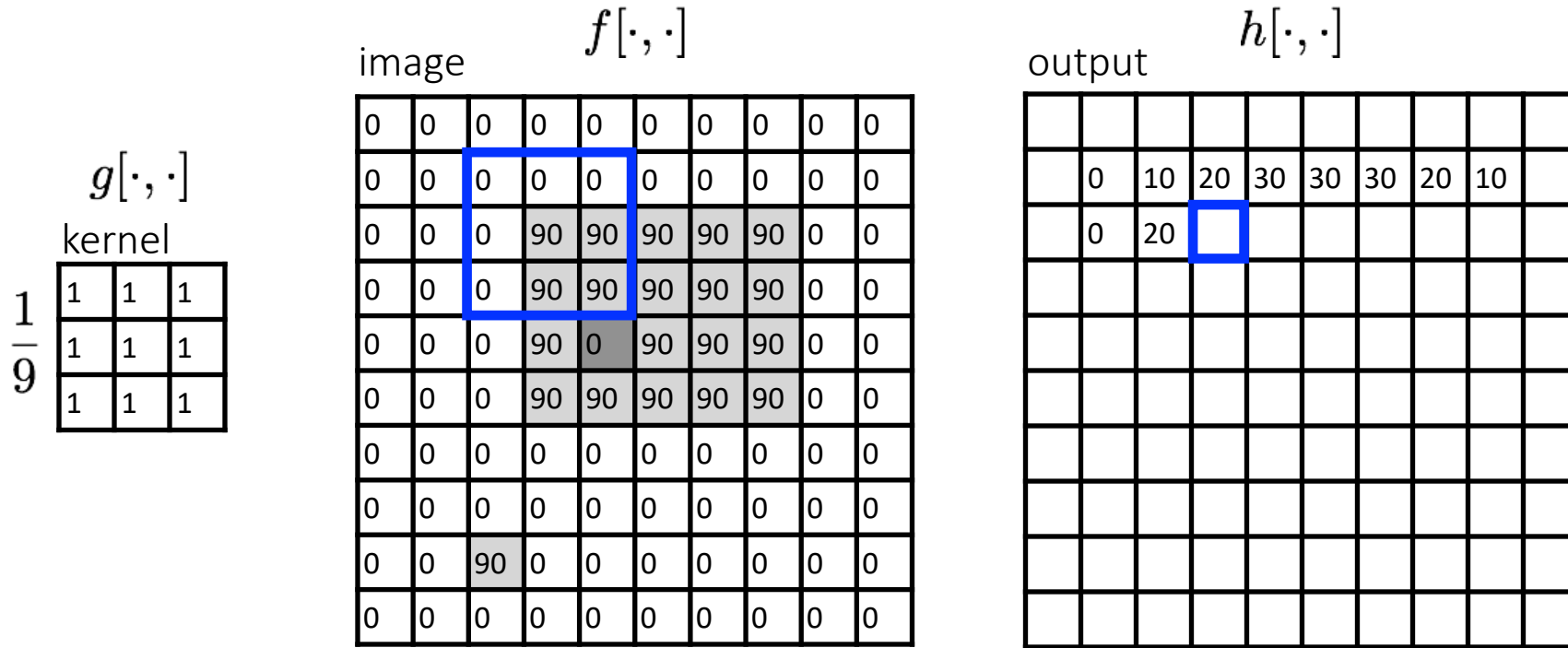
output $h[\cdot, \cdot]$

	0	10	20	30	30	30	20	10	
	0								

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output
 k, l
filter
image (signal)

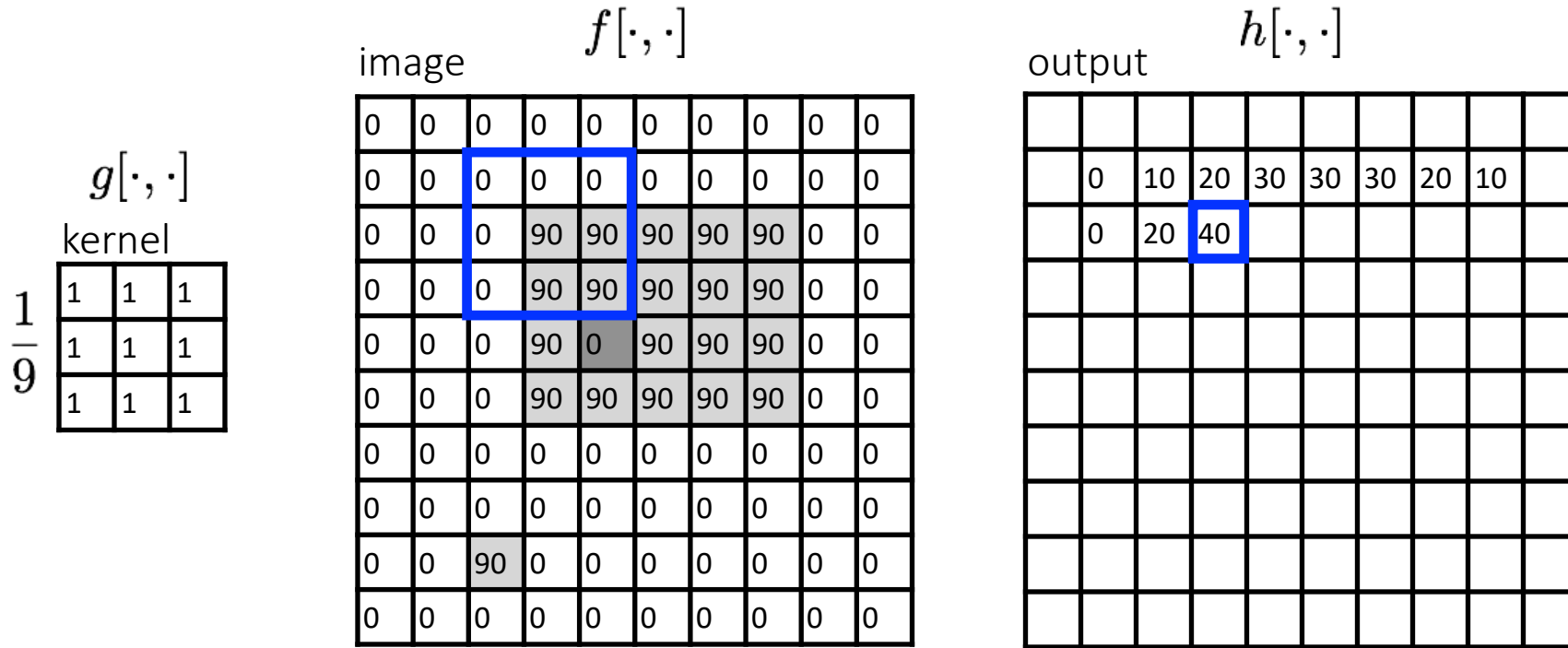
Let's run the box filter



$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output k, l filter image (signal)

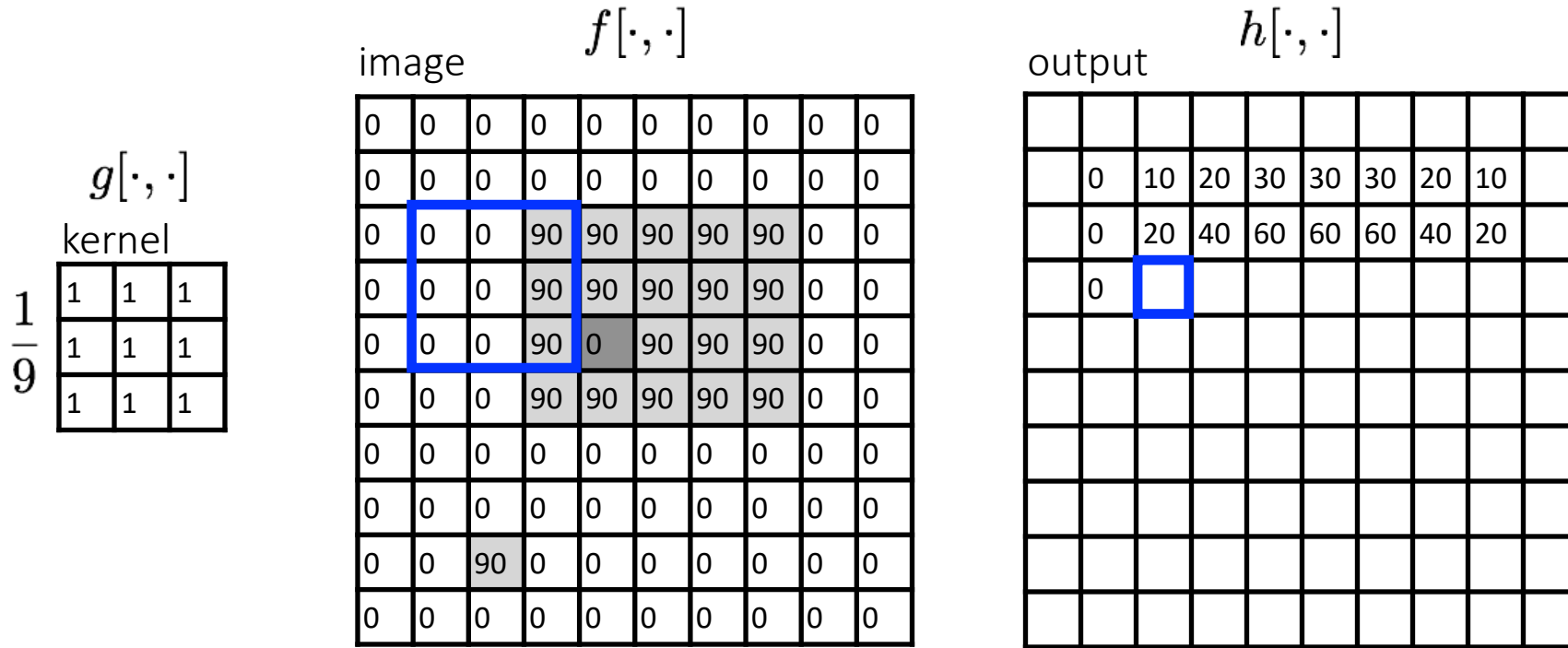
Let's run the box filter



$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output
 k, l
filter
image (signal)

Let's run the box filter



$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output
 k, l
filter
image (signal)

Let's run the box filter

$$\frac{1}{9} g[\cdot, \cdot]$$

1	1	1
1	1	1
1	1	1

image $f[\cdot, \cdot]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

output $h[\cdot, \cdot]$

	0	10	20	30	30	30	20	10	
	0	20	40	60	60	60	40	20	
	0	30							

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output
 k, l
filter
image (signal)

Let's run the box filter

$$\frac{1}{9} \begin{matrix} & & g[\cdot, \cdot] \\ & \text{kernel} & \\ \begin{matrix} \boxed{1} & \boxed{1} & \boxed{1} \\ \boxed{1} & \boxed{1} & \boxed{1} \\ \boxed{1} & \boxed{1} & \boxed{1} \end{matrix} \end{matrix}$$
[illegible][illegible]

$$\underset{\text{output}}{h[m, n]} = \sum_{k, l} \underset{\text{filter}}{g[k, l]} \underset{\text{image (signal)}}{f[m + k, n + l]}$$

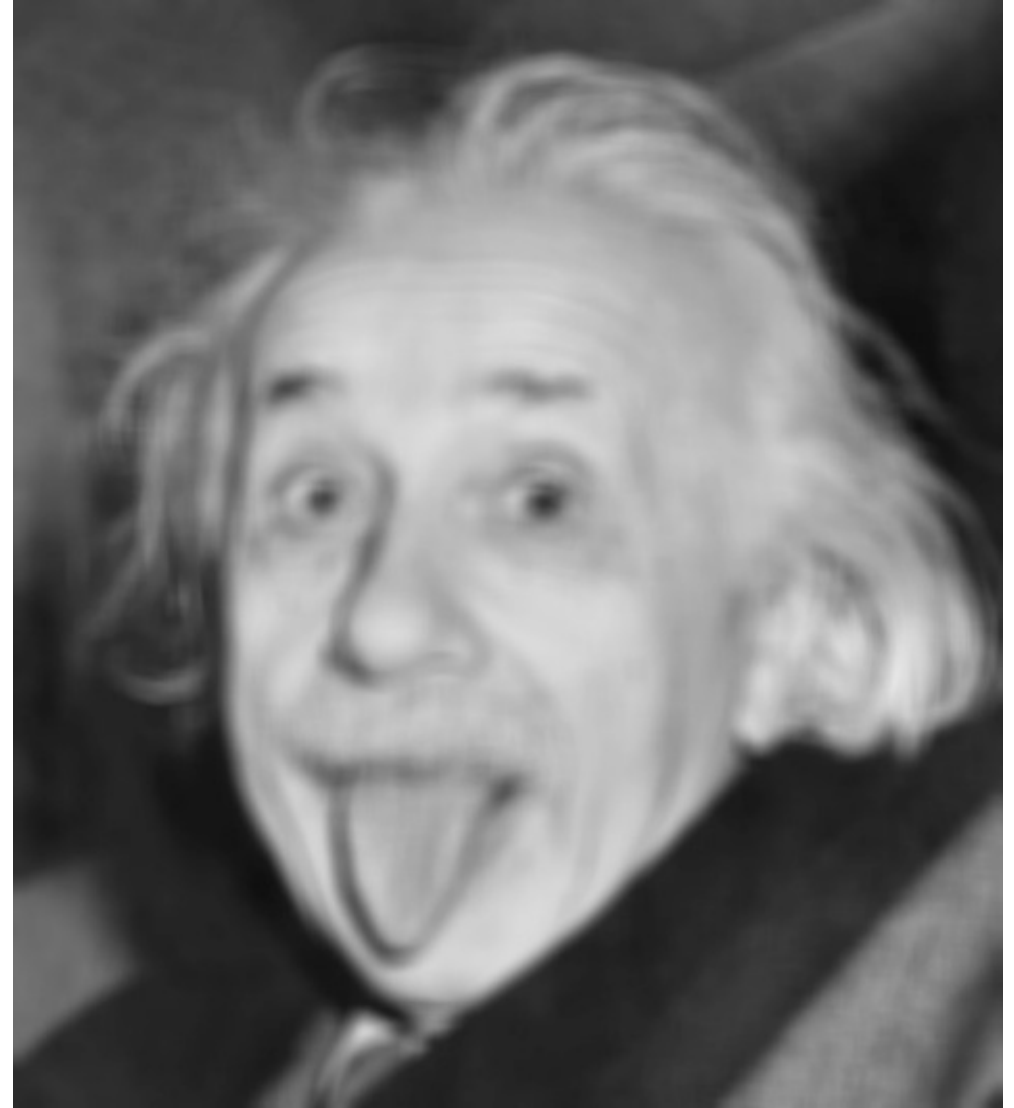
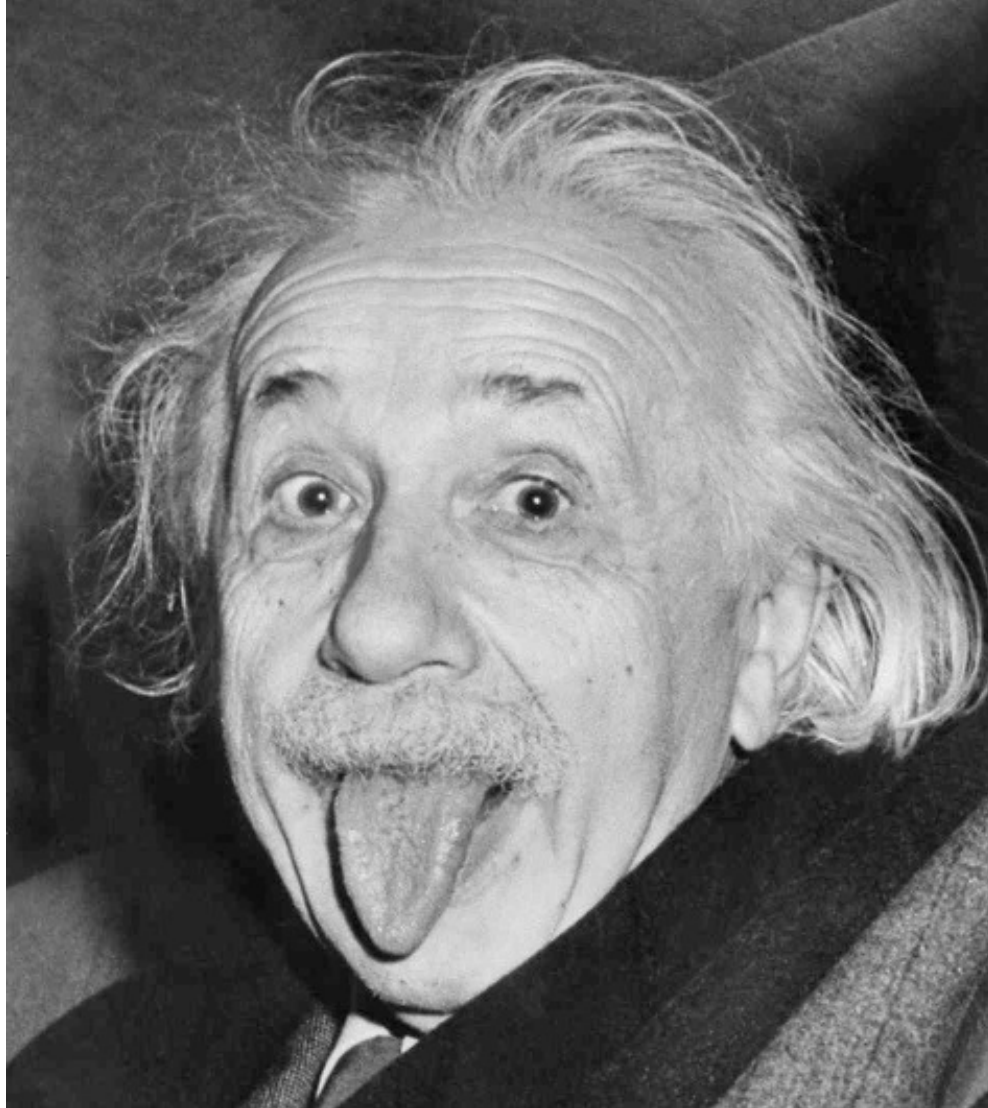
Let's run the box filter

$$\frac{1}{9} \begin{matrix} & & g[\cdot, \cdot] \\ & \text{kernel} & \\ \begin{matrix} 1 \\ 1 \\ 1 \end{matrix} & \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \end{matrix}$$
[illegible][illegible]

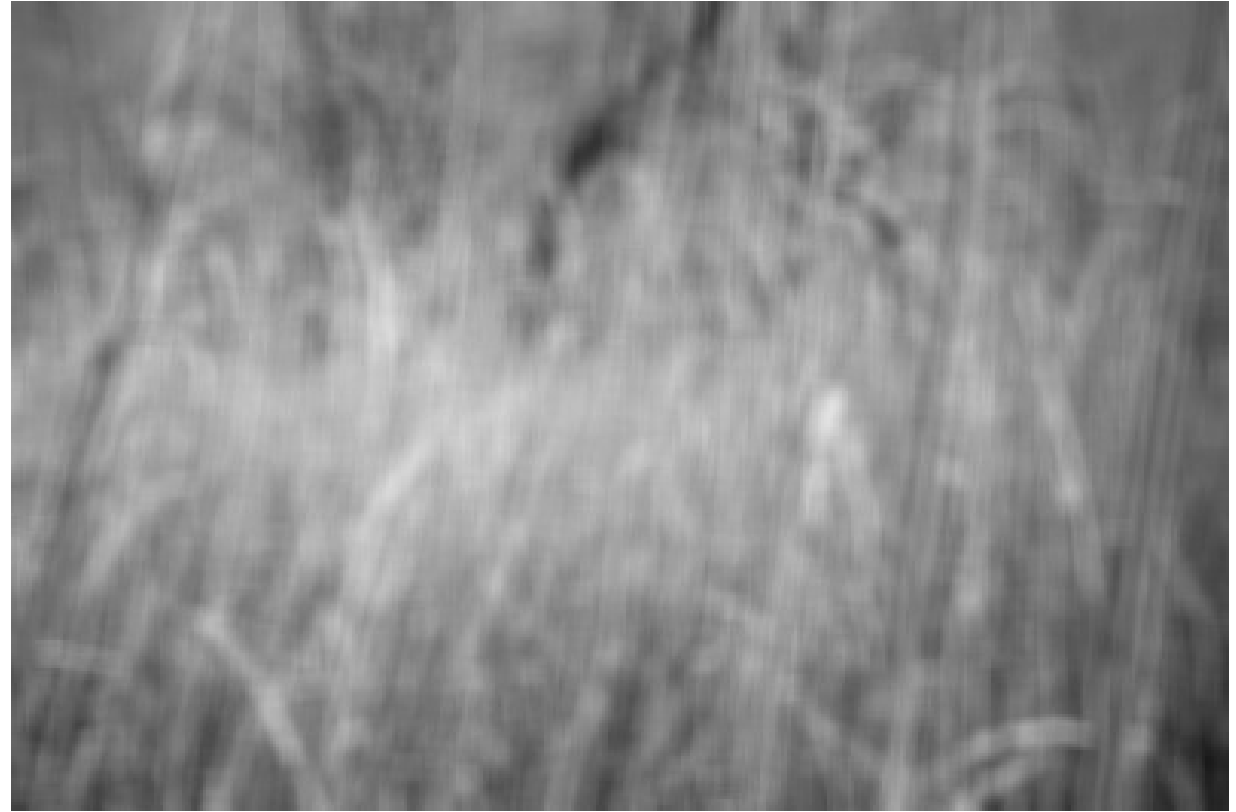
$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output
filter
image (signal)

Some more realistic examples



Some more realistic examples



Some more realistic examples



Warping

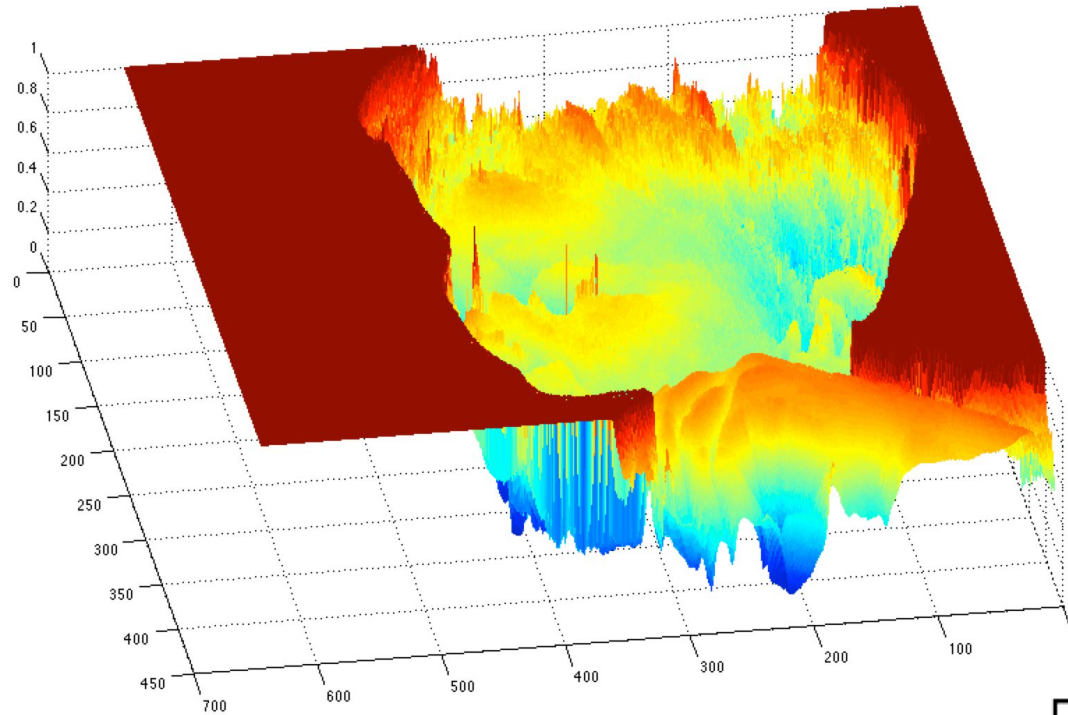
What is an image?



grayscale image

What is the range of
the image function f ?

$f(\mathbf{x})$



domain $\mathbf{x} = \begin{bmatrix} x \\ y \end{bmatrix}$

A (grayscale)
image is a 2D
function.

What types of image transformations can we do?



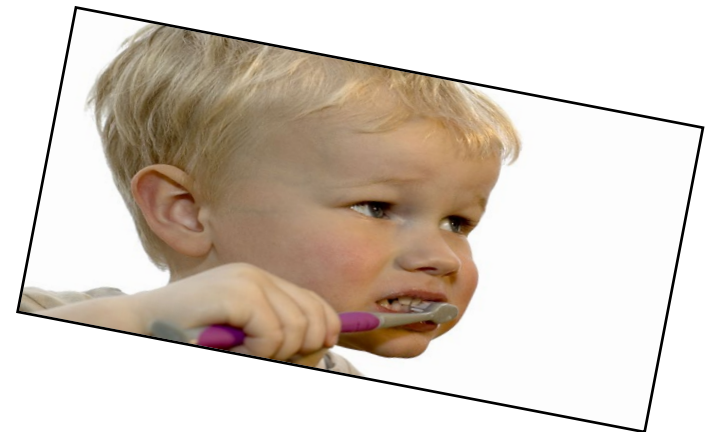
Filtering



changes pixel *values*



Warping



changes pixel *locations*

What types of image transformations can we do?

F

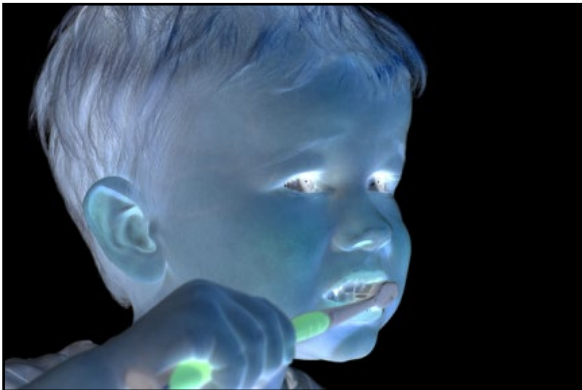


Filtering



$$G(\mathbf{x}) = h\{F(\mathbf{x})\}$$

G



changes *range* of image function

F

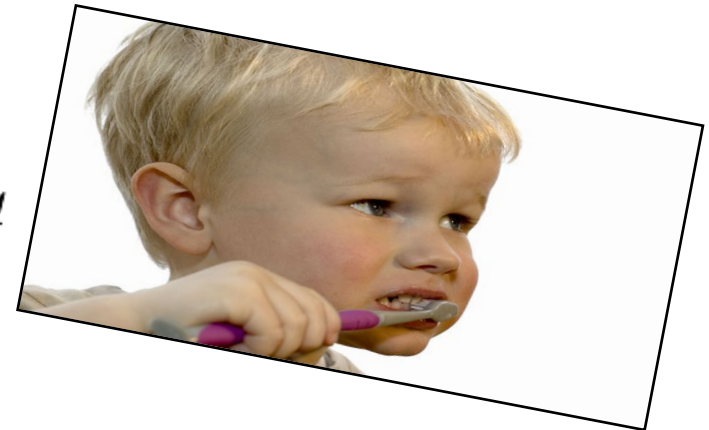


Warping



$$G(\mathbf{x}) = F(h\{\mathbf{x}\})$$

G



changes *domain* of image function

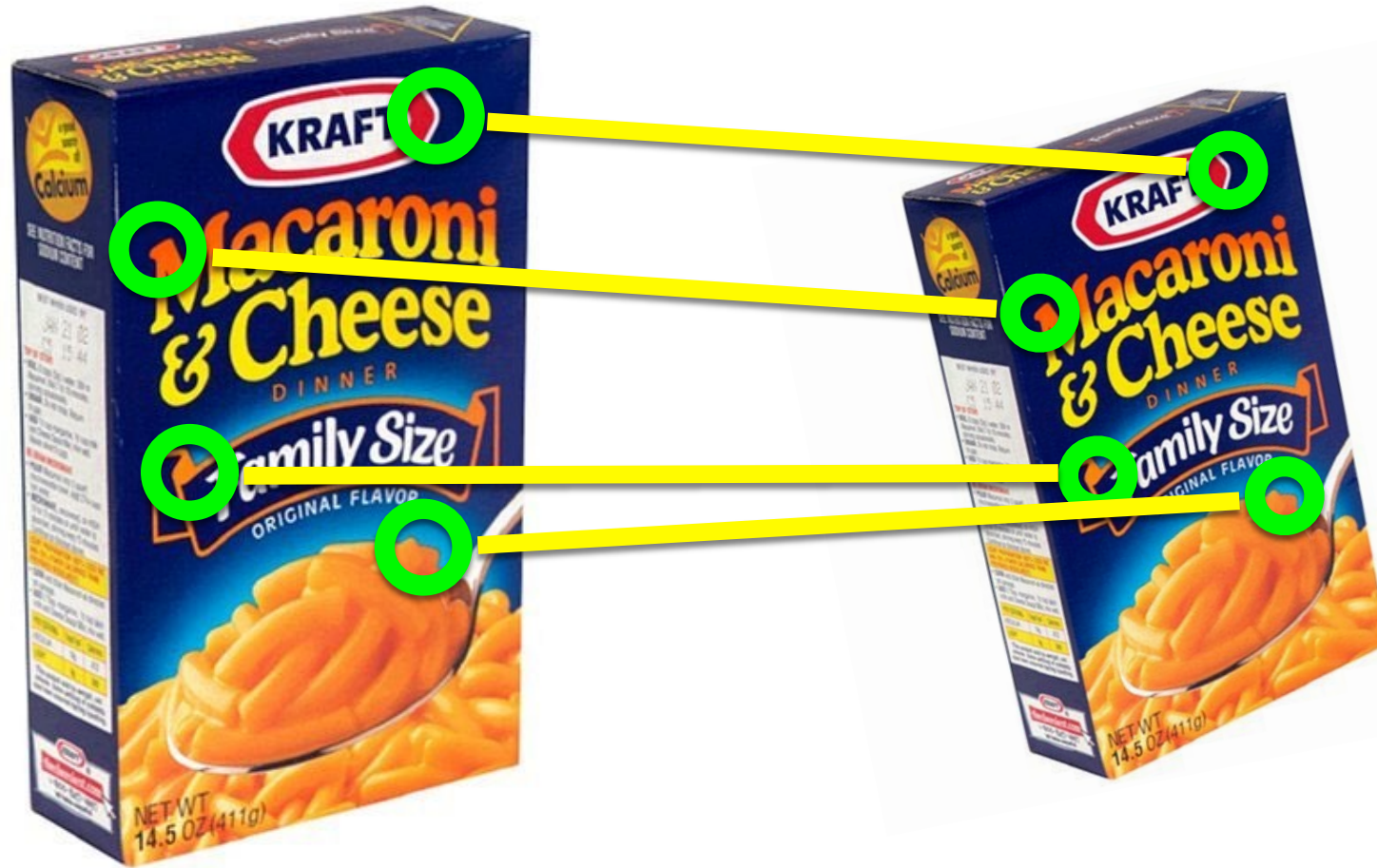
Warping example: feature matching



Warping example: feature matching



Warping example: feature matching



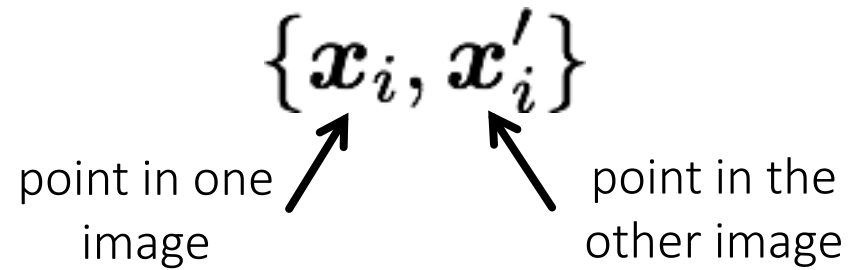
- object recognition
- 3D reconstruction
- augmented reality
- image stitching

缝合

How do you compute the transformation?

Warping example: feature matching

Given a set of matched feature points:



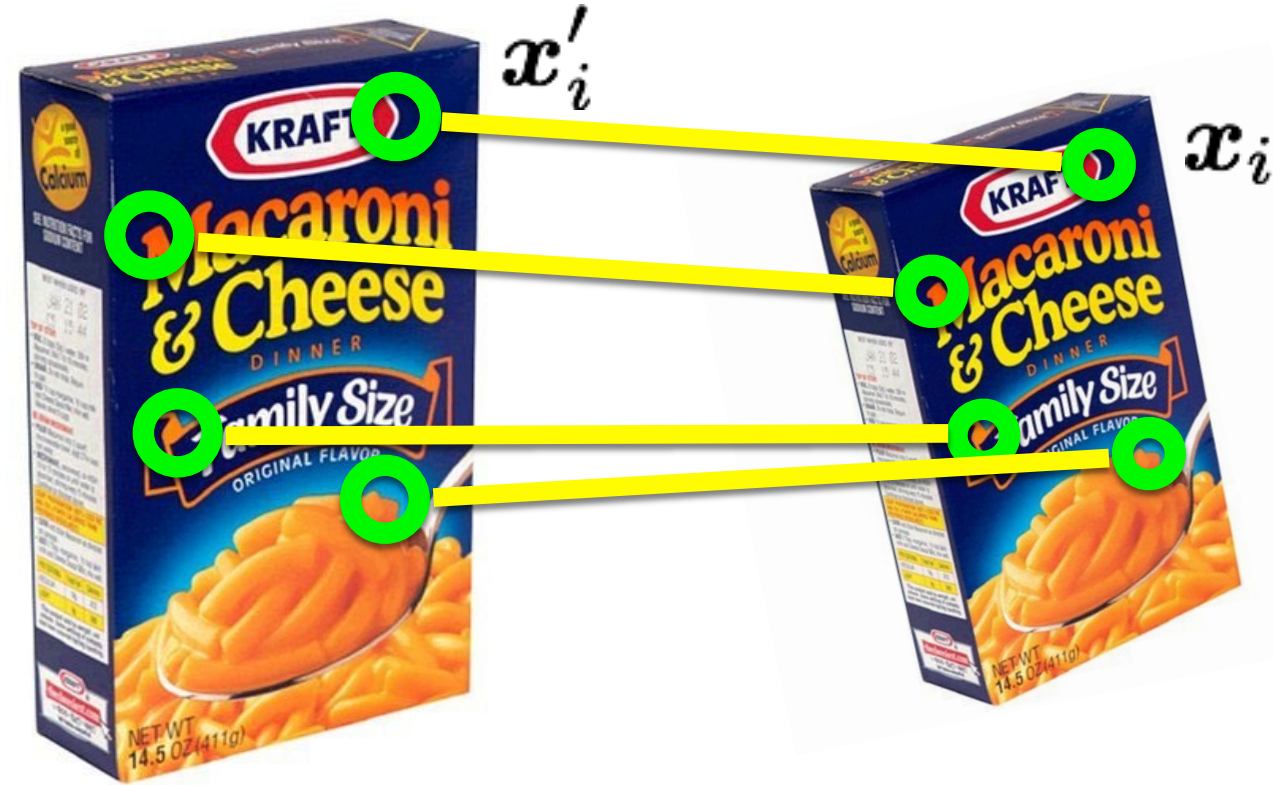
and a transformation:

$$x' = f(x; p)$$

transformation function parameters

find the best estimate of the parameters

p



What kind of transformation functions f are there?

2D transformations

2D transformations



translation



rotation



aspect



affine



perspective

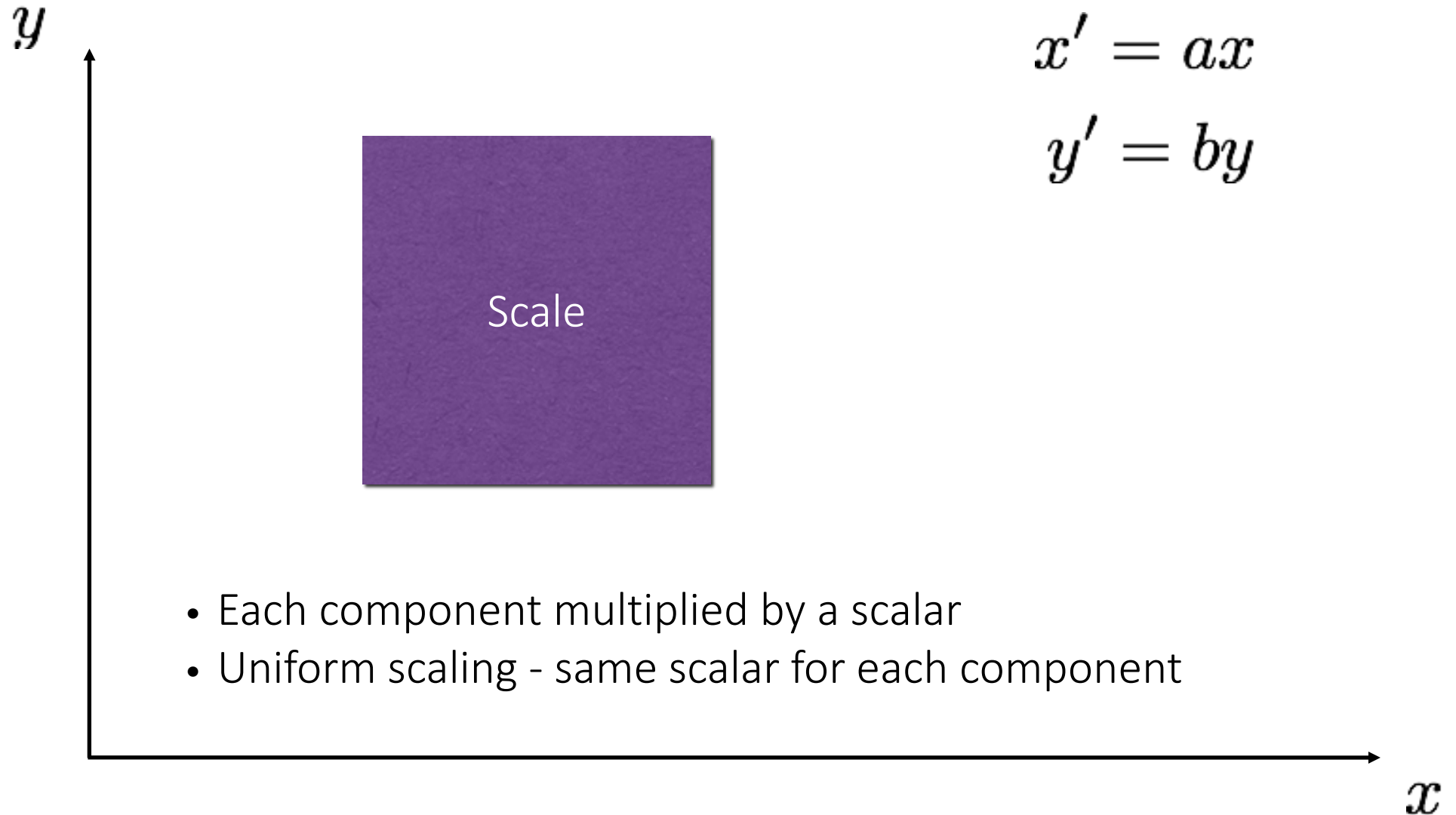


cylindrical

2D planar transformations



2D planar transformations



2D planar transformations

y



$$x' = ax$$

$$y' = by$$

matrix representation of scaling:

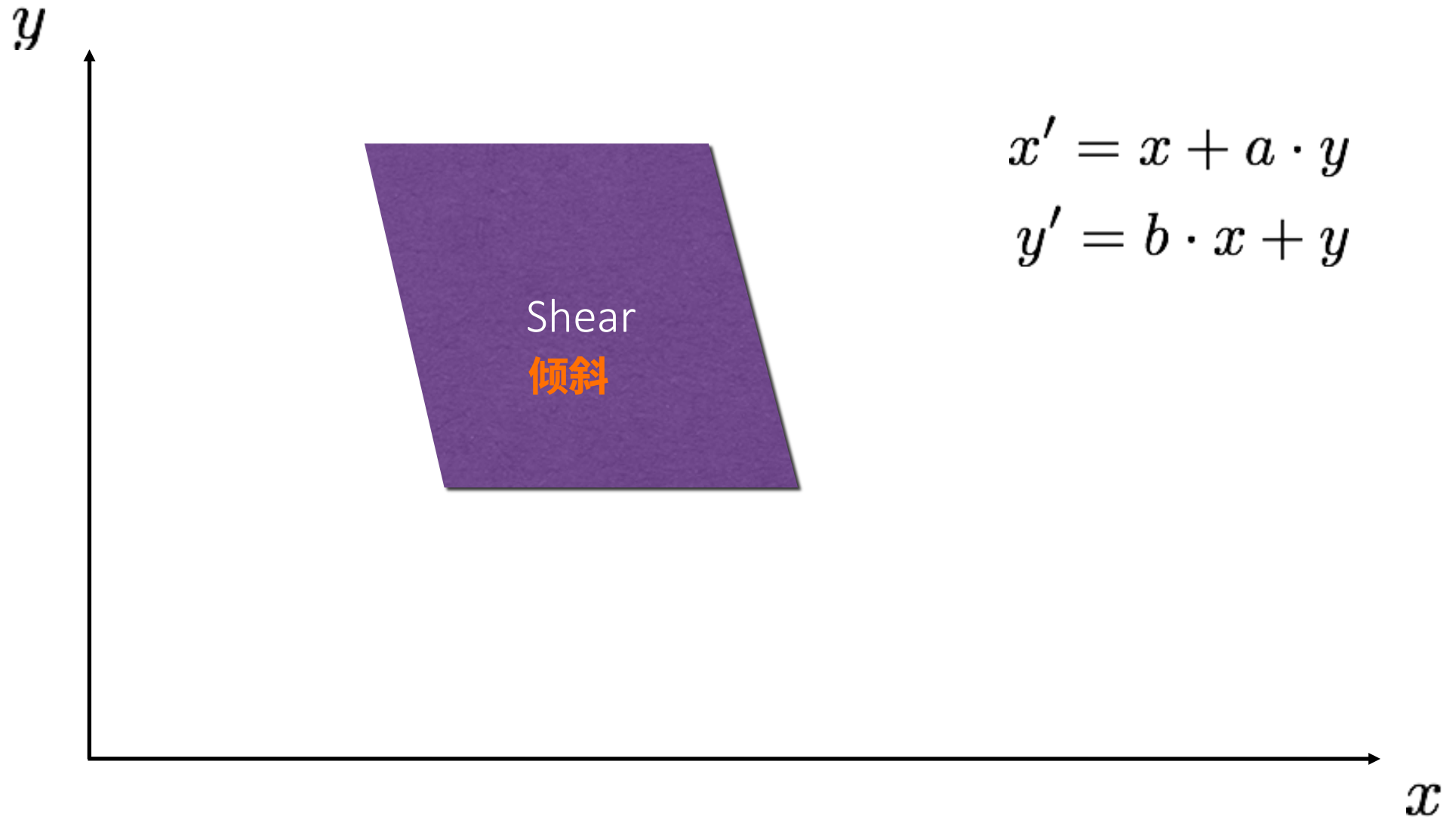
$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \underbrace{\begin{bmatrix} a & 0 \\ 0 & b \end{bmatrix}}_{\text{scaling matrix } S} \begin{bmatrix} x \\ y \end{bmatrix}$$

scaling matrix S

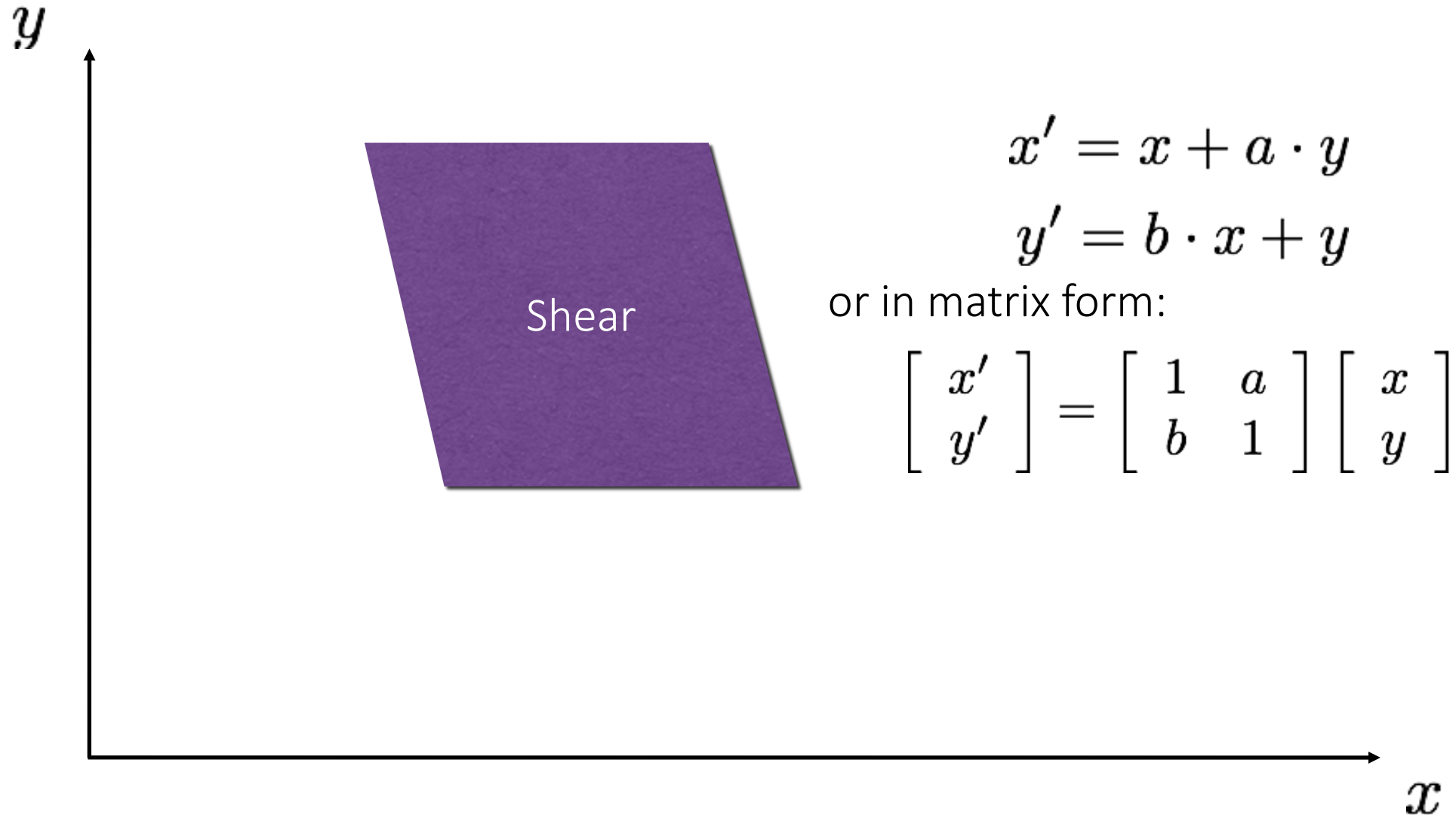
- Each component multiplied by a scalar
- Uniform scaling - same scalar for each component

x

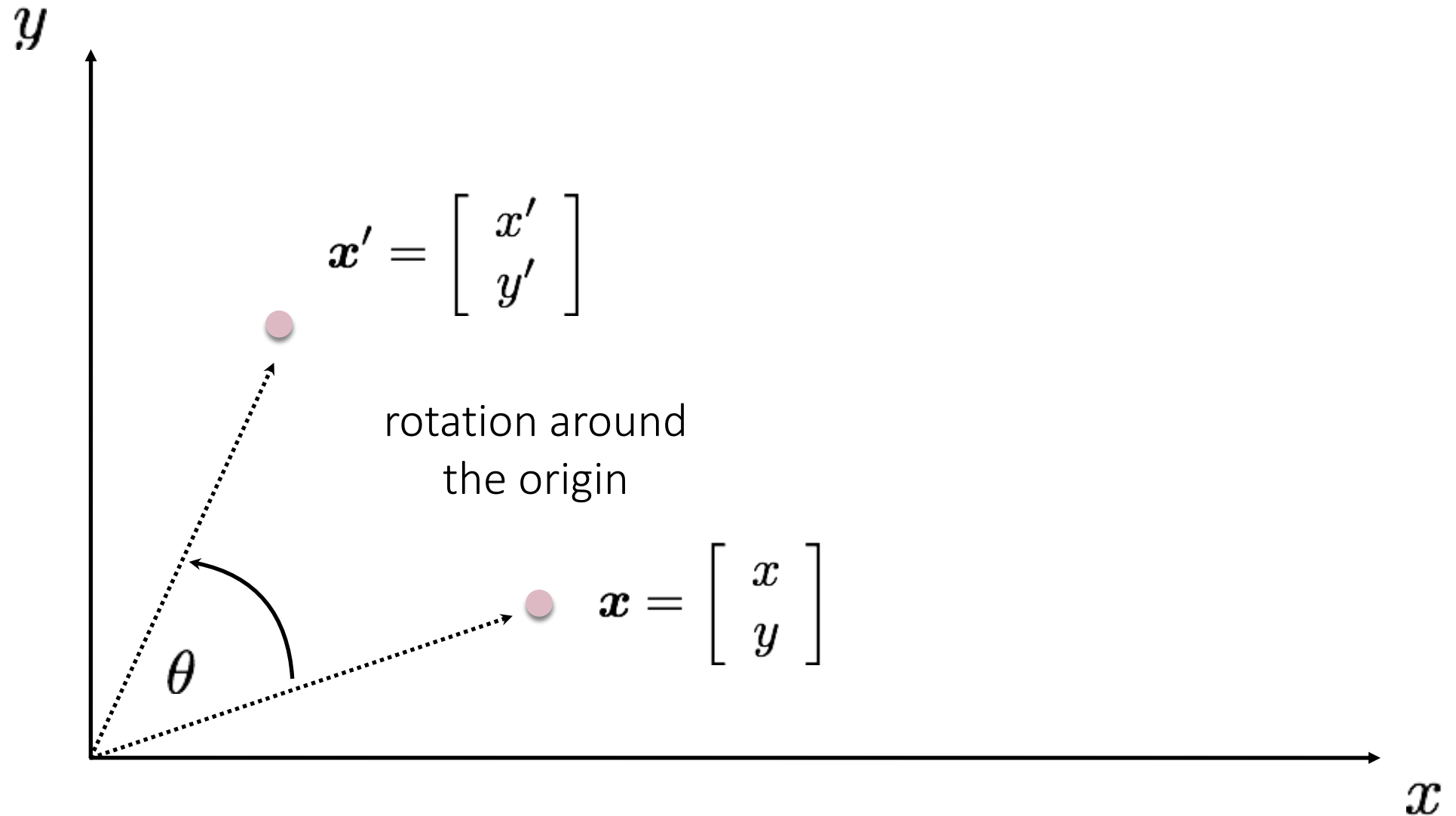
2D planar transformations



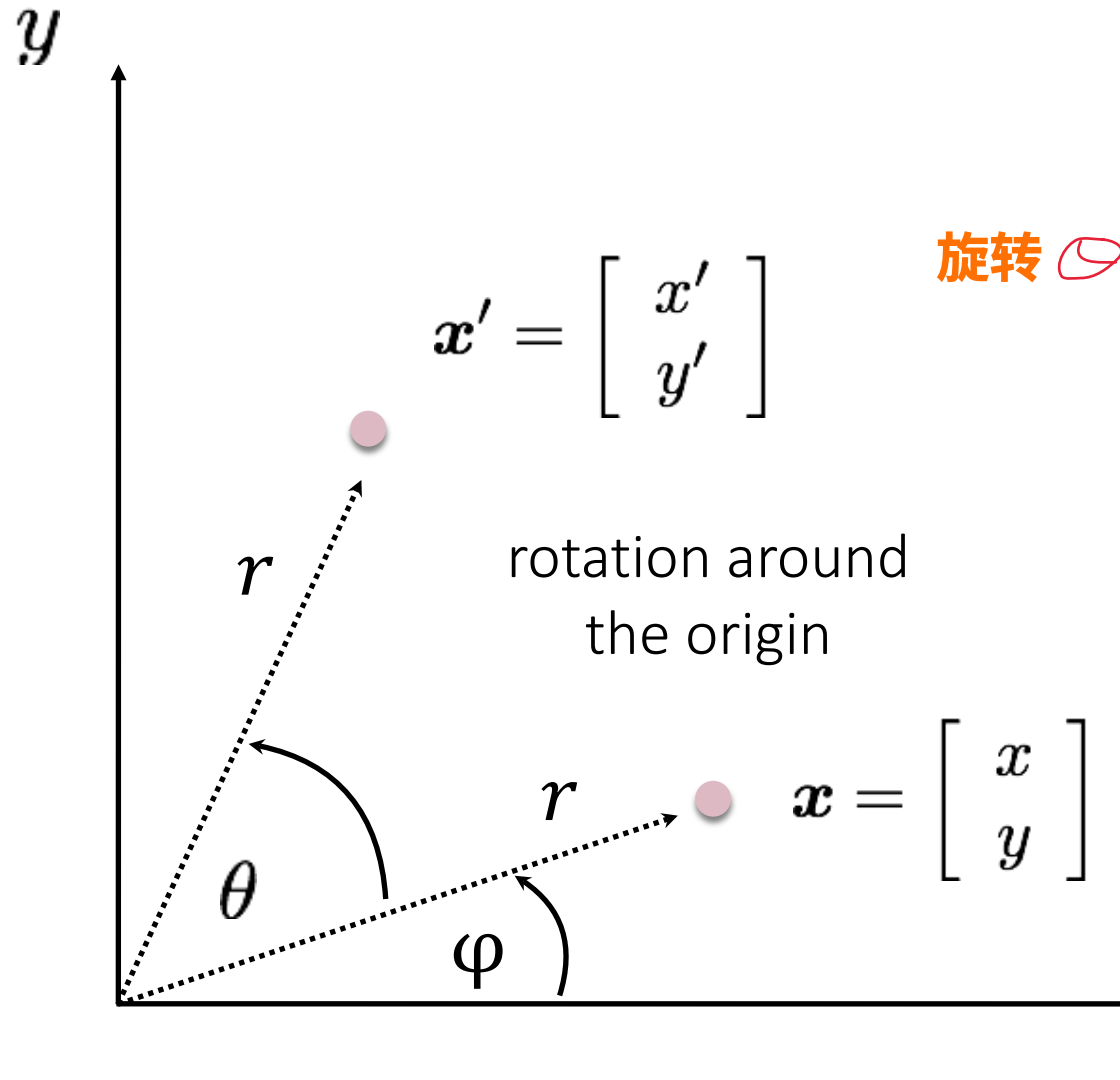
2D planar transformations



2D planar transformations



2D planar transformations



Polar coordinates...

$$x = r \cos(\phi)$$

$$y = r \sin(\phi)$$

$$x' = r \cos(\phi + \theta)$$

$$y' = r \sin(\phi + \theta)$$

Trigonometric Identity...

$$x' = r \cos(\phi) \cos(\theta) - r \sin(\phi) \sin(\theta)$$

$$y' = r \sin(\phi) \cos(\theta) + r \cos(\phi) \sin(\theta)$$

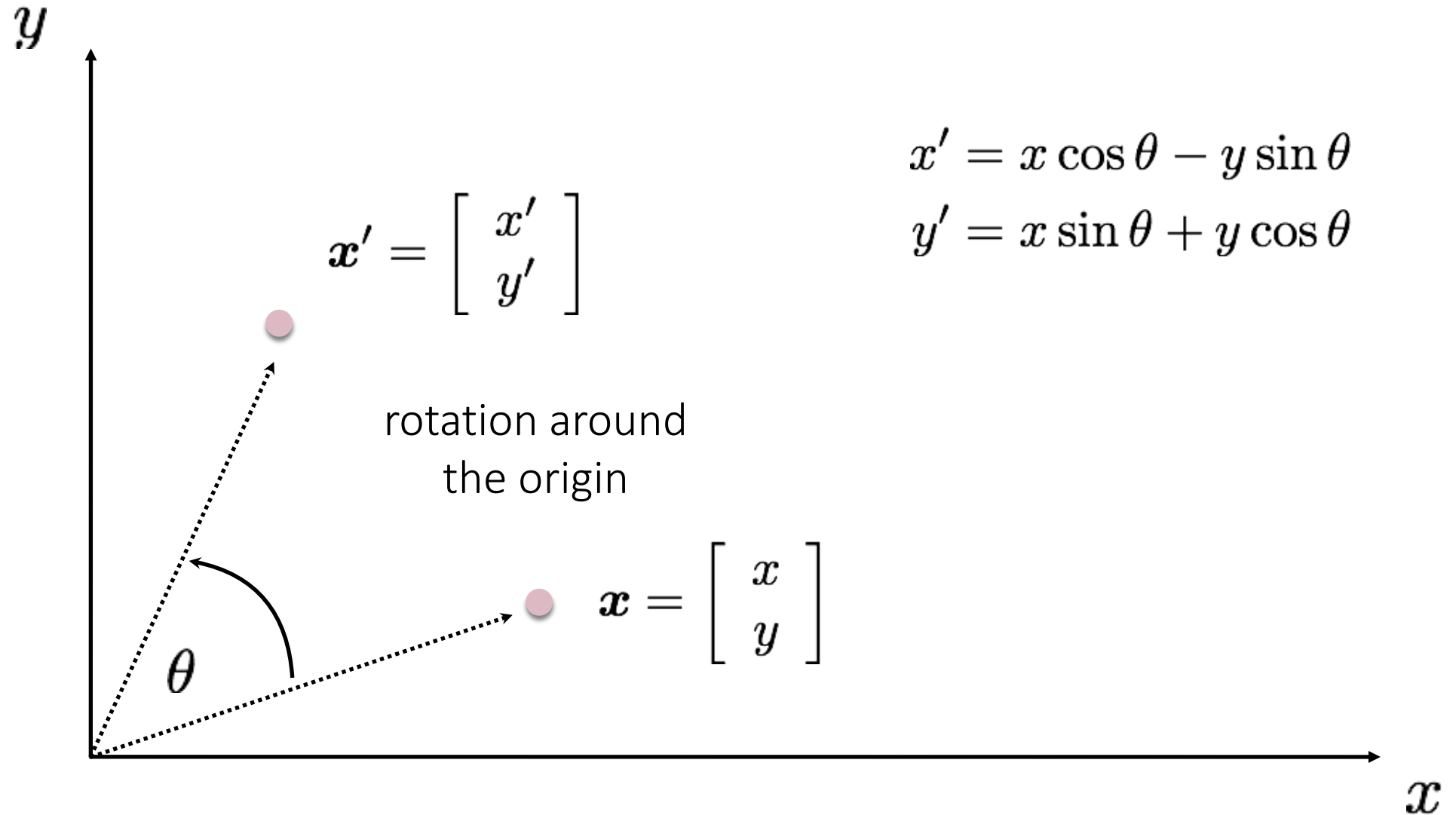
Substitute...

$$x' = x \cos(\theta) - y \sin(\theta)$$

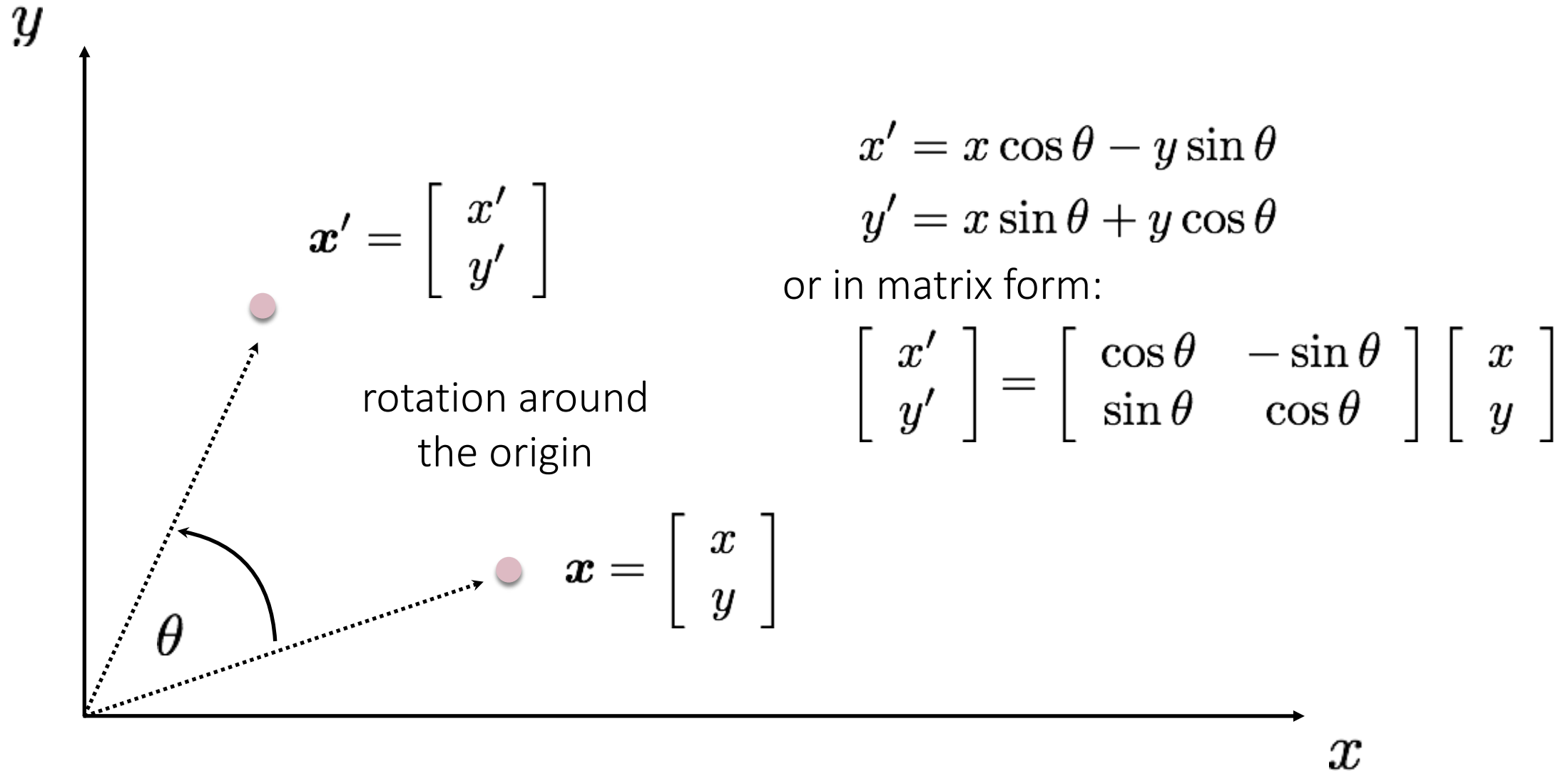
$$y' = x \sin(\theta) + y \cos(\theta)$$

x

2D planar transformations



2D planar transformations



2D planar and linear transformations

$$\boldsymbol{x}' = f(\boldsymbol{x}; p)$$



$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \boldsymbol{M} \begin{bmatrix} x \\ y \end{bmatrix}$$

parameters p

point $\boldsymbol{x}$

2D planar and linear transformations

Scale

$$\mathbf{M} = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix}$$

Flip across y

$$\mathbf{M} = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}$$

Rotate

$$\mathbf{M} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

Flip across origin

$$\mathbf{M} = \begin{bmatrix} -1 & 0 \\ 0 & -1 \end{bmatrix}$$

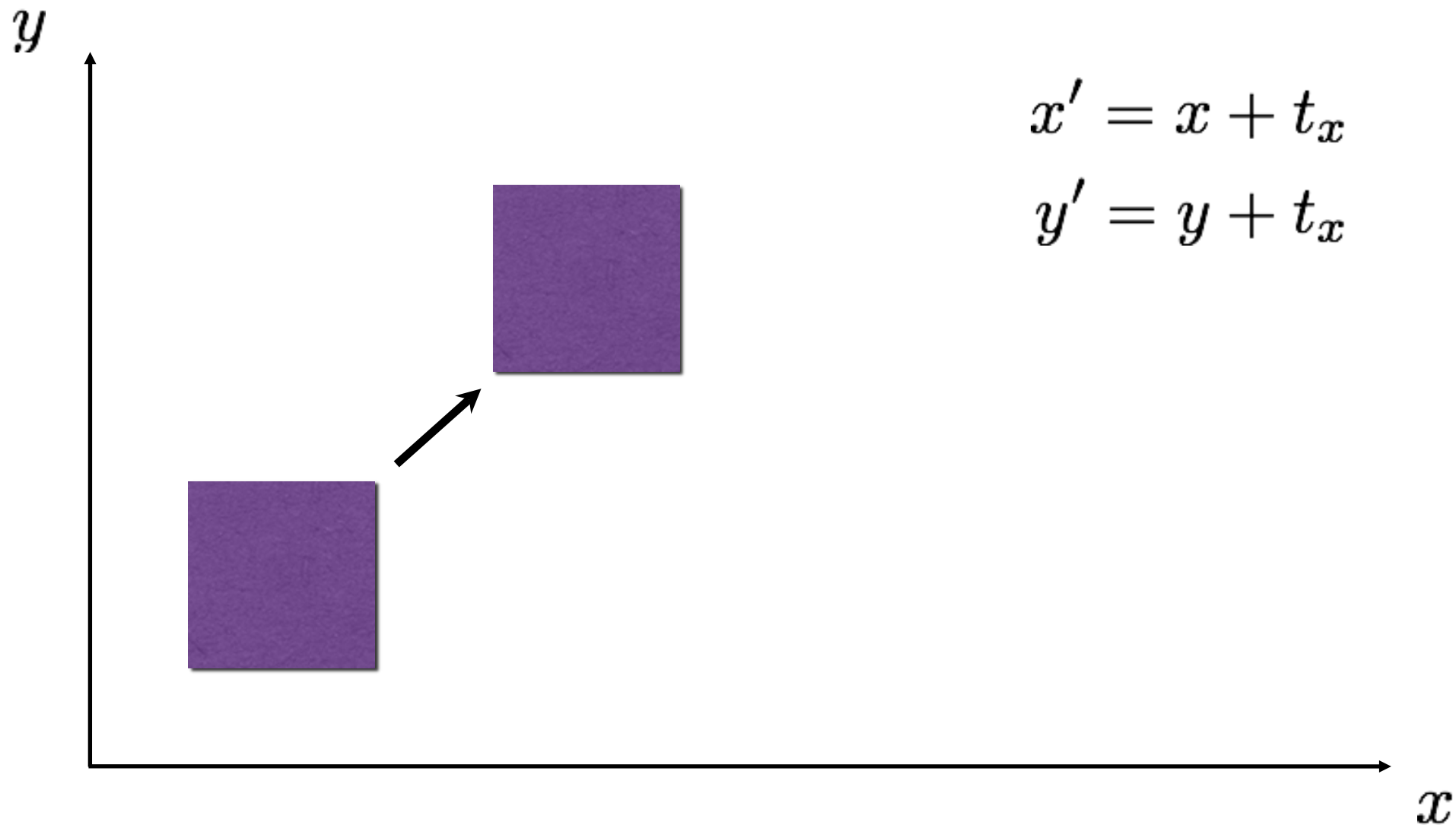
Shear

$$\mathbf{M} = \begin{bmatrix} 1 & s_x \\ s_y & 1 \end{bmatrix}$$

Identity

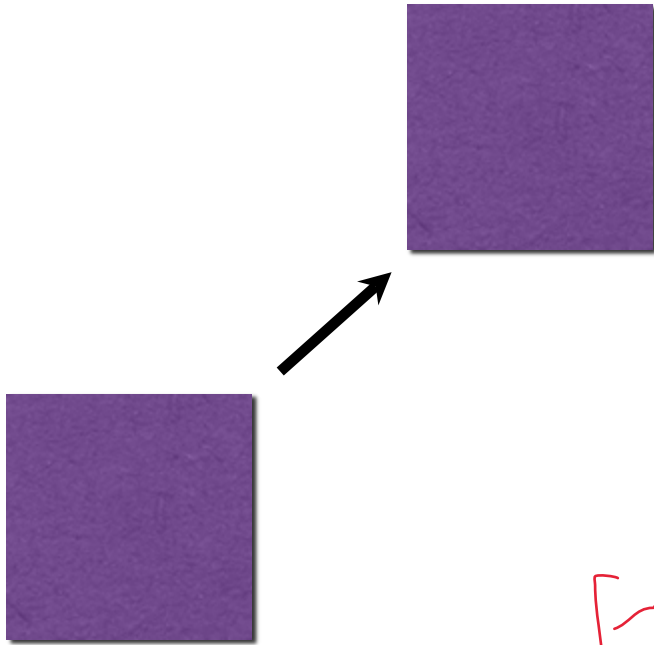
$$\mathbf{M} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

2D translation



2D translation

y



$$x' = x + t_x$$

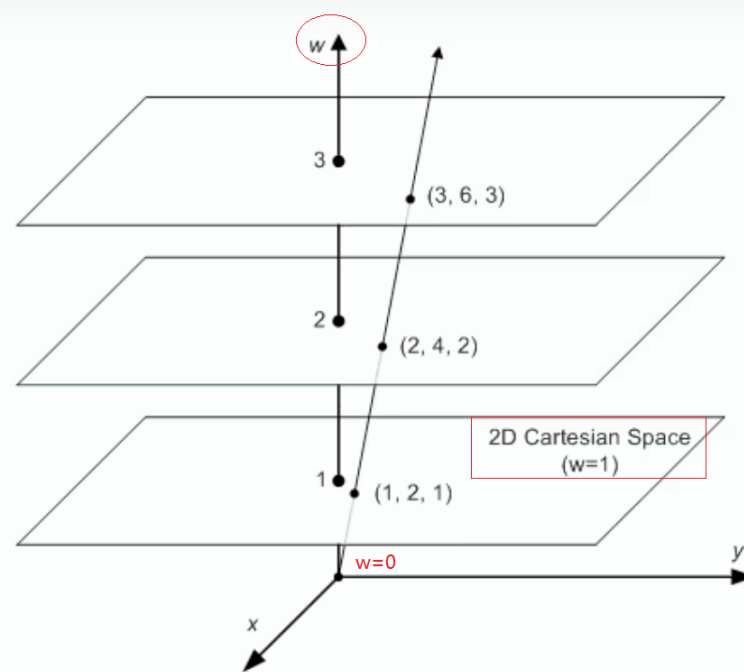
$$y' = y + t_y$$

What about matrix representation?

$$\mathbf{M} = \begin{bmatrix} ? & ? \\ ? & ? \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

x



Homogeneous coordinates

齐次坐标就是将一个原本是 n 维的向量用一个 $n+1$ 维向量来表示，是指一个用于投影几何里的坐标系，如同用于欧氏几何里的笛卡儿坐标一般

https://zhuanlan.zhihu.com/p/625678401?utm_id=0

<https://learnopencv.com/homography-examples-using-opencv-python-c/>

2D homogeneous coordinates

heterogeneous
coordinates

homogeneous
coordinates

$$\begin{bmatrix} x \\ y \end{bmatrix} \Rightarrow \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

add a 1 here



- Represent 2D point with a 3D vector

2D homogeneous coordinates

heterogeneous
coordinates

homogeneous
coordinates

$$\begin{bmatrix} x \\ y \end{bmatrix} \Rightarrow \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \stackrel{\text{def}}{=} \begin{bmatrix} ax \\ ay \\ a \end{bmatrix}$$

- Represent 2D point with a 3D vector
- 3D vectors are only defined up to scale

仅按比例定义

2D homogeneous coordinates

Conversion:

- heterogeneous $\rightarrow$ homogeneous

$$\begin{bmatrix} x \\ y \end{bmatrix} \Rightarrow \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Scale invariance:

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix} = a \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

2D homogeneous coordinates

Conversion:

- heterogeneous $\rightarrow$ homogeneous

$$\begin{bmatrix} x \\ y \end{bmatrix} \Rightarrow \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- homogeneous $\rightarrow$ heterogeneous

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix} \Rightarrow \begin{bmatrix} x/z \\ y/z \end{bmatrix}$$

Scale invariance:

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix} = a \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

2D homogeneous coordinates

Conversion:

- heterogeneous $\rightarrow$ homogeneous

$$\begin{bmatrix} x \\ y \end{bmatrix} \Rightarrow \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- homogeneous $\rightarrow$ heterogeneous

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix} \Rightarrow \begin{bmatrix} x/z \\ y/z \end{bmatrix}$$

z 越小，越靠近原点

Scale invariance:

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix} = a \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

Special points:

- point at infinity

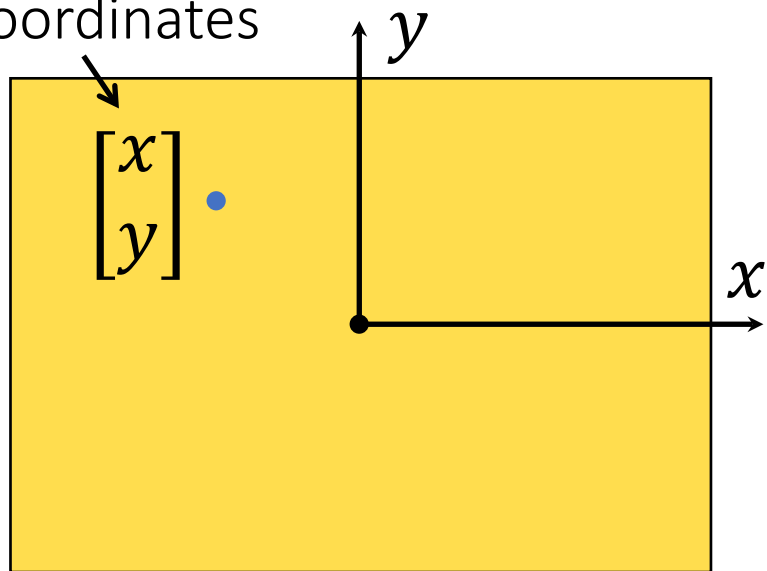
$$\begin{bmatrix} x \\ y \\ 0 \end{bmatrix}$$

- undefined

$$\begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

2D projective geometry

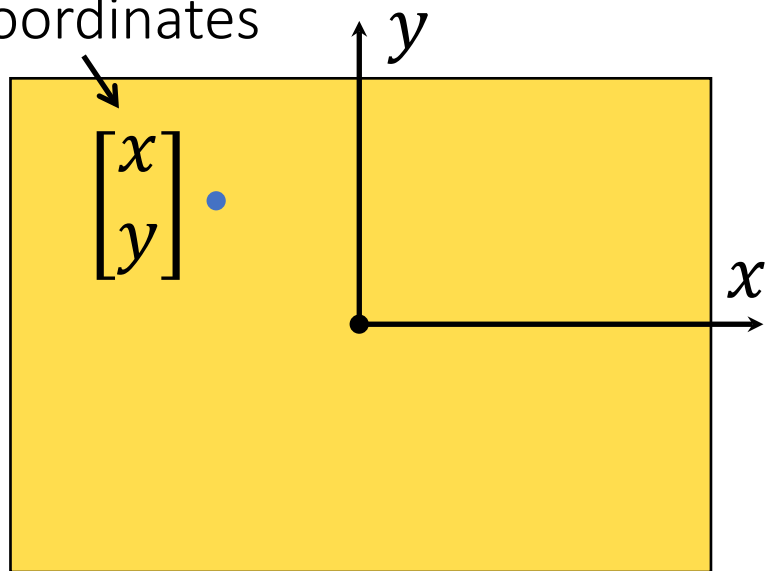
heterogeneous
coordinates



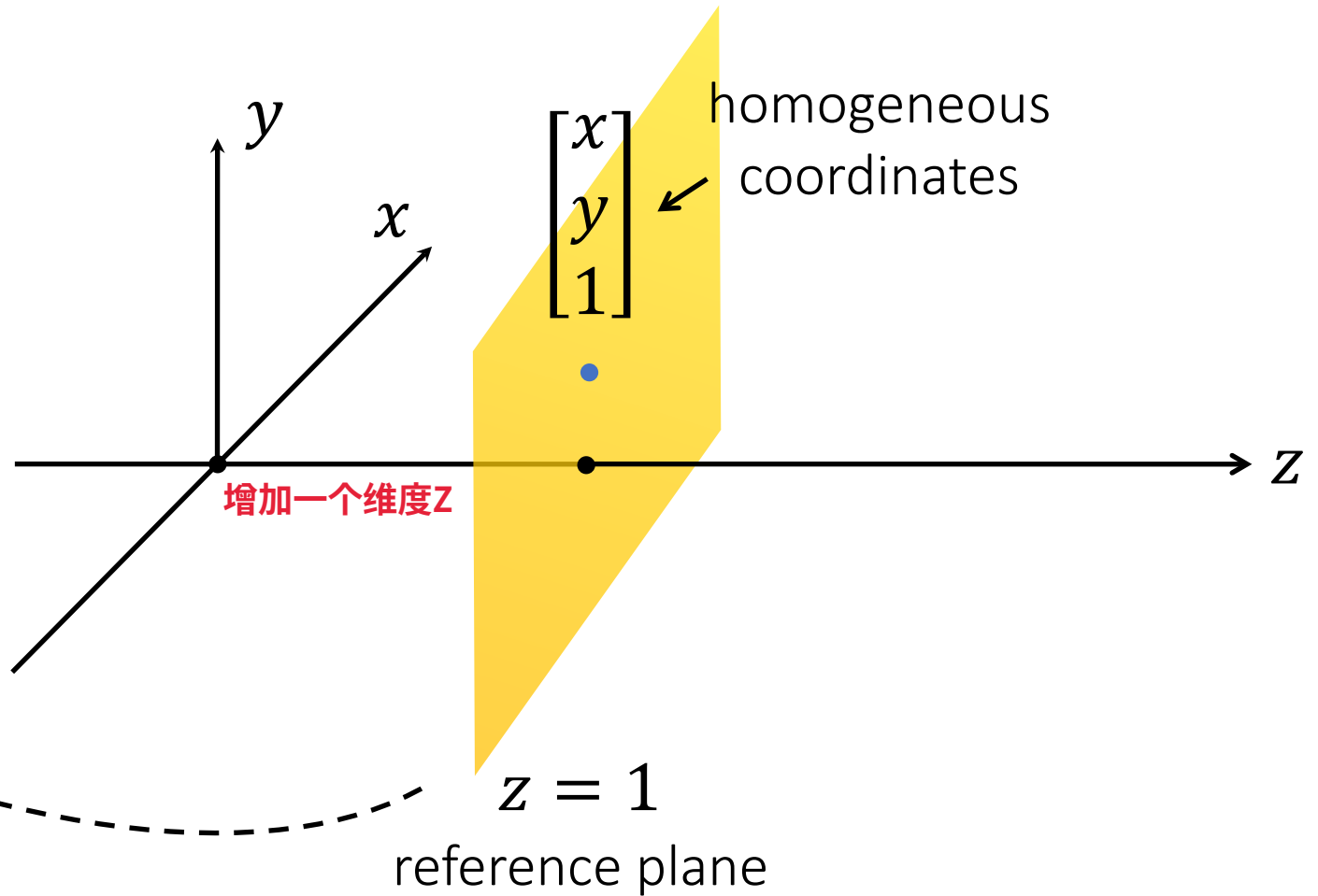
reference plane

2D projective geometry

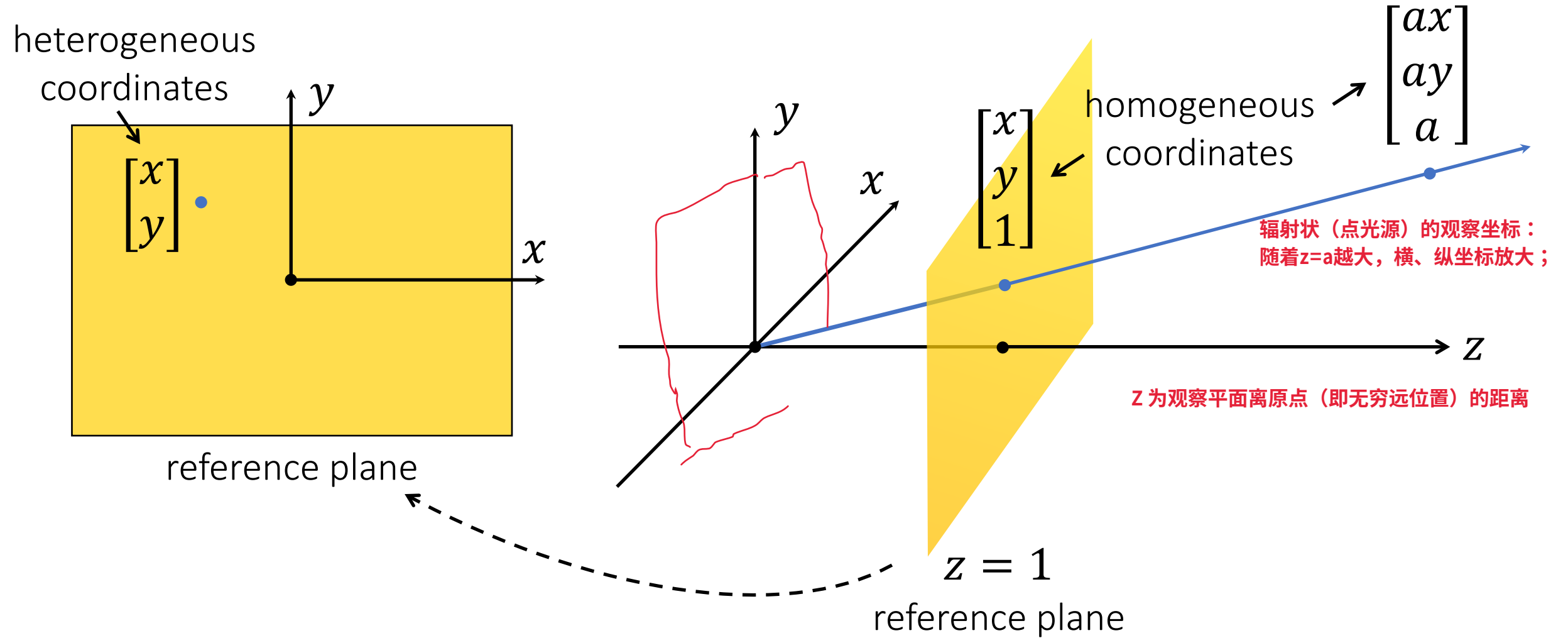
heterogeneous
coordinates



reference plane



2D projective geometry



Through the scale invariance property, homogeneous coordinates map all points on a line passing through the origin to the point where this line intersects the reference plane.

3D homogeneous coordinates

heterogeneous
coordinates

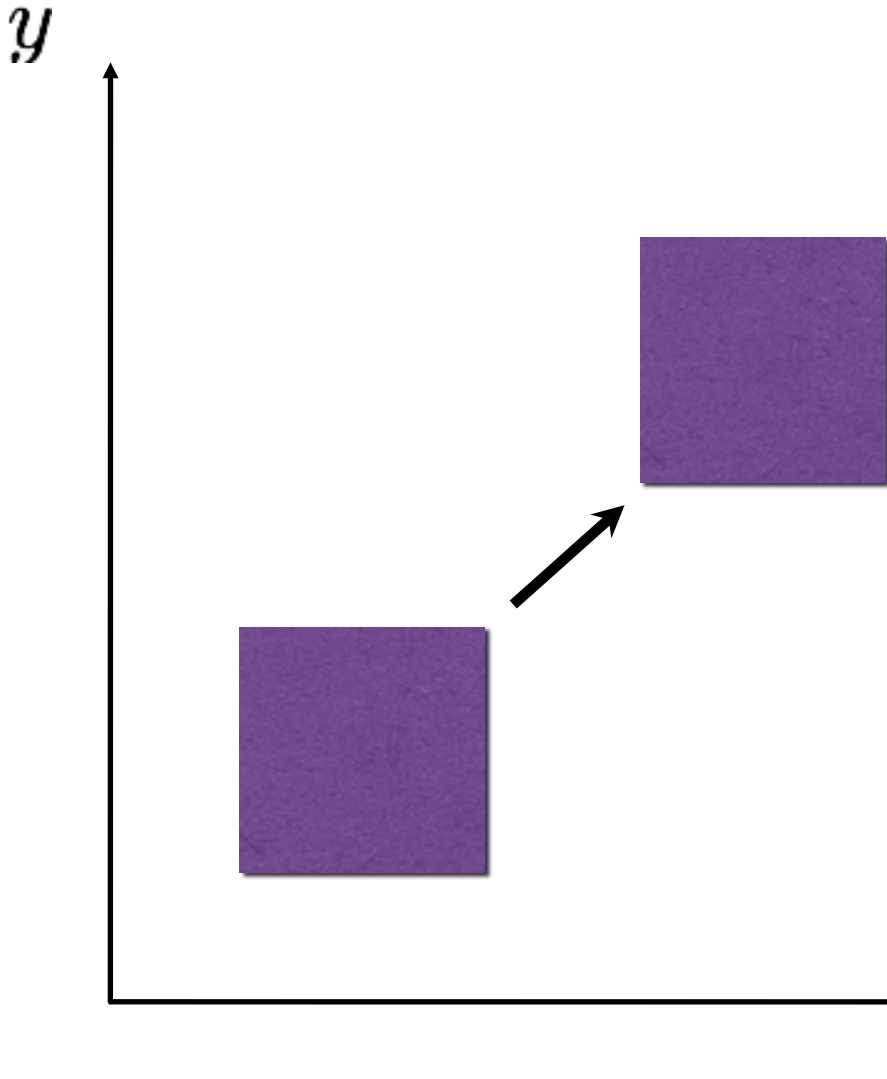
homogeneous
coordinates

$$\begin{bmatrix} X \\ Y \\ Z \end{bmatrix} \Rightarrow \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \stackrel{\text{def}}{=} \begin{bmatrix} aX \\ aY \\ aZ \\ a \end{bmatrix}$$

- Represent 3D point with a 4D vector
- 4D vectors are only defined up to scale

2D transformations in homogeneous coordinates

2D translation

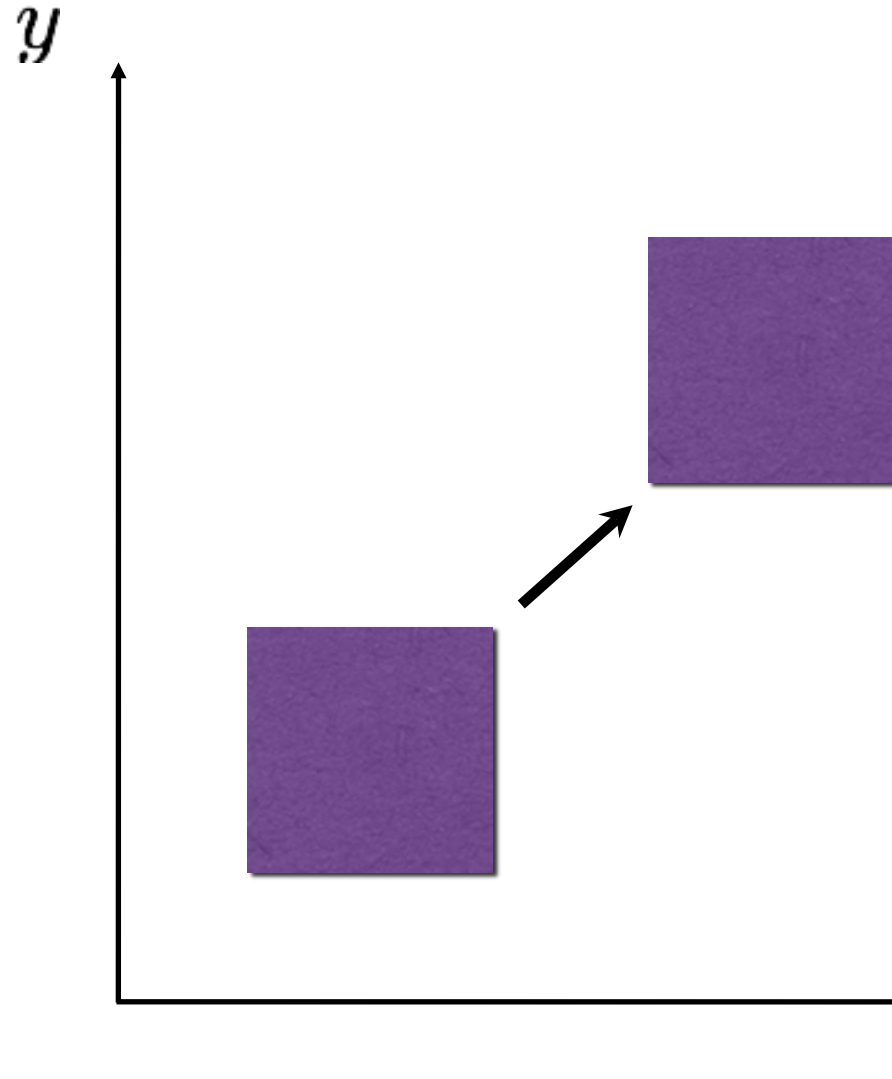


$$x' = x + t_x$$

$$y' = y + t_y$$

What about matrix representation
using homogeneous coordinates?

2D translation in homogeneous coordinates



$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x + t_x \\ y + t_y \\ 1 \end{bmatrix}$$

What about matrix representation using homogeneous coordinates?

$$\mathbf{M} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

2D transformations in homogeneous coordinates

Re-write these transformations as 3x3 matrices:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

translation

scaling

rotation

shearing

2D transformations in homogeneous coordinates

Re-write these transformations as 3x3 matrices:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

translation

rotation

$$\begin{bmatrix} \mathbf{x}' \\ \mathbf{y}' \\ 1 \end{bmatrix} = \begin{bmatrix} \mathbf{s}_x & 0 & 0 \\ 0 & \mathbf{s}_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \mathbf{x} \\ \mathbf{y} \\ 1 \end{bmatrix}$$

scaling

shearing

2D transformations in homogeneous coordinates

Re-write these transformations as 3x3 matrices:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

translation

$$\begin{bmatrix} \mathbf{x}' \\ \mathbf{y}' \\ 1 \end{bmatrix} = \begin{bmatrix} \mathbf{s}_x & 0 & 0 \\ 0 & \mathbf{s}_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \mathbf{x} \\ \mathbf{y} \\ 1 \end{bmatrix}$$

scaling

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & \beta_x & 0 \\ \beta_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

rotation

shearing

2D transformations in homogeneous coordinates

Re-write these transformations as 3x3 matrices:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

translation

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

scaling

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \Theta & -\sin \Theta & 0 \\ \sin \Theta & \cos \Theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

rotation

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & \beta_x & 0 \\ \beta_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

shearing

Matrix composition

Transformations can be combined by matrix multiplication:

$$\begin{bmatrix} x' \\ y' \\ \underline{w'} \end{bmatrix} = \left(\begin{bmatrix} 1 & 0 & tx \\ 0 & 1 & ty \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos \Theta & -\sin \Theta & 0 \\ \sin \Theta & \cos \Theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} sx & 0 & 0 \\ 0 & sy & 0 \\ 0 & 0 & 1 \end{bmatrix} \right) \begin{bmatrix} x \\ y \\ \underline{w} \end{bmatrix}$$

$\mathbf{p'}$ = translation(t_x, t_y) rotation(θ) scale(s, s) $\mathbf{p}$

Matrix composition

Transformations can be combined by matrix multiplication:

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \left(\begin{bmatrix} 1 & 0 & tx \\ 0 & 1 & ty \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos \Theta & -\sin \Theta & 0 \\ \sin \Theta & \cos \Theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} sx & 0 & 0 \\ 0 & sy & 0 \\ 0 & 0 & 1 \end{bmatrix} \right) \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

$\mathbf{p}' = \text{translation}(t_x, t_y) \quad \text{rotation}(\theta) \quad \text{scale}(s, s) \quad \mathbf{p}$

Multiplication order matters! Transformations are applied from right to left.

Matrix composition

Transformations can be combined by matrix multiplication:

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \left(\begin{bmatrix} 1 & 0 & tx \\ 0 & 1 & ty \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos \Theta & -\sin \Theta & 0 \\ \sin \Theta & \cos \Theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} sx & 0 & 0 \\ 0 & sy & 0 \\ 0 & 0 & 1 \end{bmatrix} \right) \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

$\mathbf{p}'$ = translation(t_x, t_y) rotation(θ) scale(s, s) $\mathbf{p}$

Multiplication order matters! Transformations are applied from right to left.

Matrix composition

Transformations can be combined by matrix multiplication:

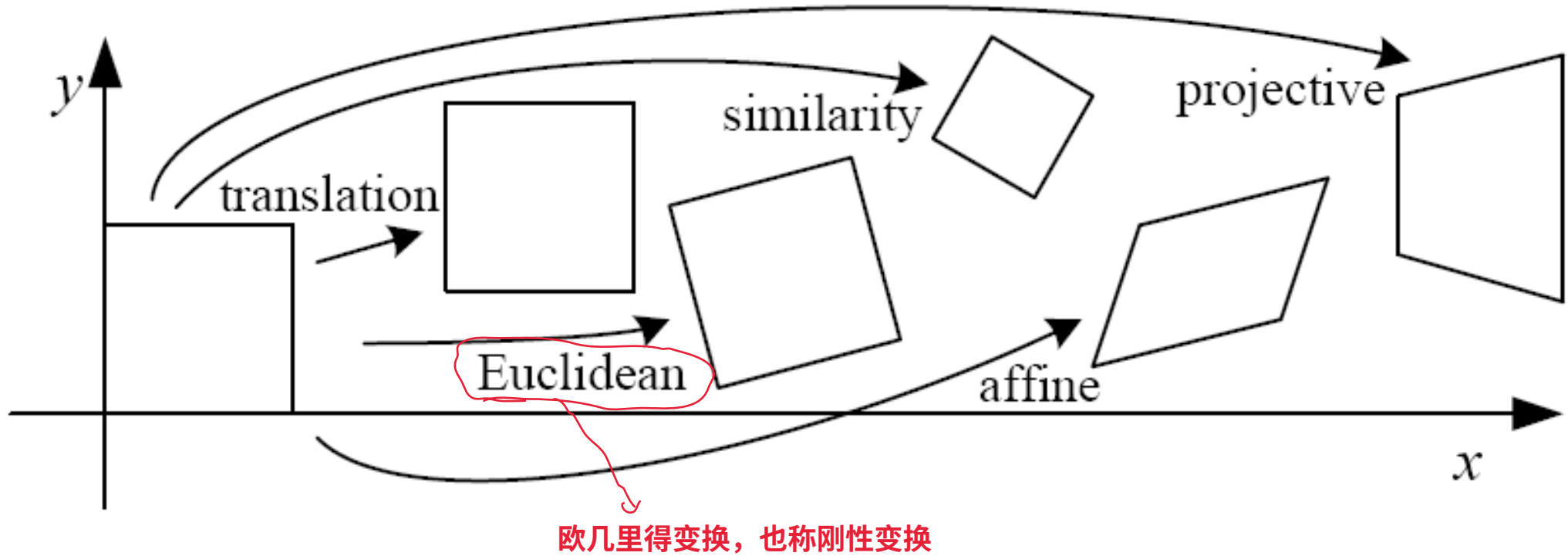
$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \left(\begin{bmatrix} 1 & 0 & tx \\ 0 & 1 & ty \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos \Theta & -\sin \Theta & 0 \\ \sin \Theta & \cos \Theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} sx & 0 & 0 \\ 0 & sy & 0 \\ 0 & 0 & 1 \end{bmatrix} \right) \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

$$\mathbf{p}' = \text{translation}(t_x, t_y) \quad \text{rotation}(\theta) \quad \text{scale}(s, s) \quad \mathbf{p}$$

Multiplication order matters! Transformations are applied from right to left.

Classification of 2D transformations

Classification of 2D transformations



刚性变换、仿射变换、透视变换和非线性变换

<https://blog.csdn.net/u013403054/article/details/107558212>

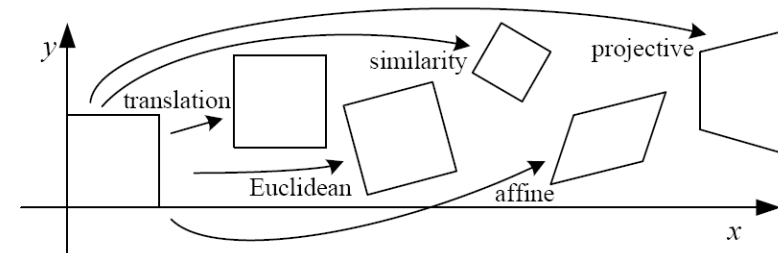
Classification of 2D transformations

Translation:

$$\begin{bmatrix} 1 & 0 & t_1 \\ 0 & 1 & t_2 \\ 0 & 0 & 1 \end{bmatrix}$$

degrees of freedom: 2

即DoF：可自由选择的参数个数

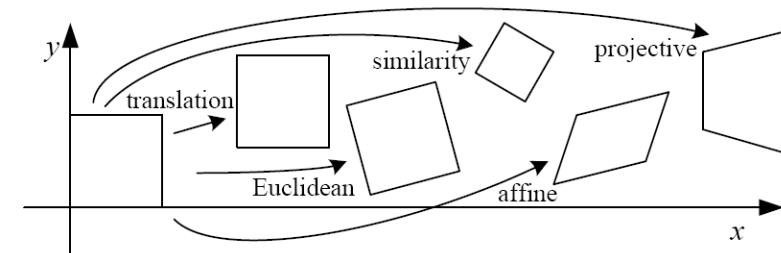


Classification of 2D transformations

a.k.a. 是 also known as
Euclidean (a.k.a. rigid):
rotation + translation

$$\begin{bmatrix} \cos \theta & -\sin \theta & t_1 \\ \sin \theta & \cos \theta & t_2 \\ 0 & 0 & 1 \end{bmatrix}$$

degrees of freedom: 3



Classification of 2D transformations

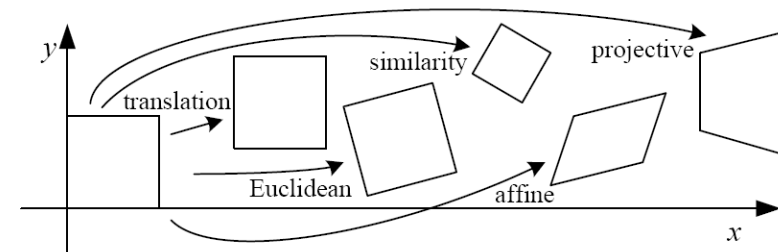
changing this matrix entry affects
three other matrix entries

3

Euclidean (a.k.a. rigid):
rotation + translation

$$\begin{bmatrix} \cos \theta & -\sin \theta & t_1 \\ \sin \theta & \cos \theta & t_2 \\ 0 & 0 & 1 \end{bmatrix}$$

degrees of freedom: 3

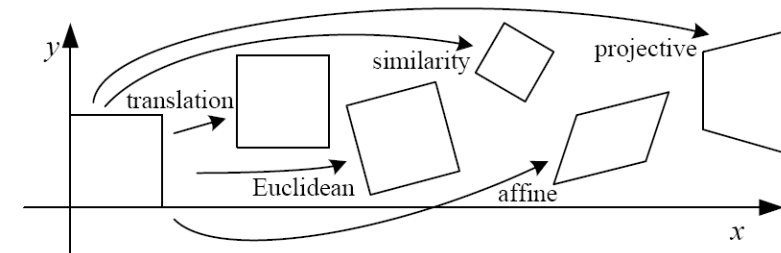


Classification of 2D transformations

Similarity:
uniform scaling + rotation
+ translation

$$\begin{bmatrix} s \cdot \cos \theta & -s \cdot \sin \theta & t_1 \\ s \cdot \sin \theta & s \cdot \cos \theta & t_2 \\ 0 & 0 & 1 \end{bmatrix}$$

degrees of freedom: 4

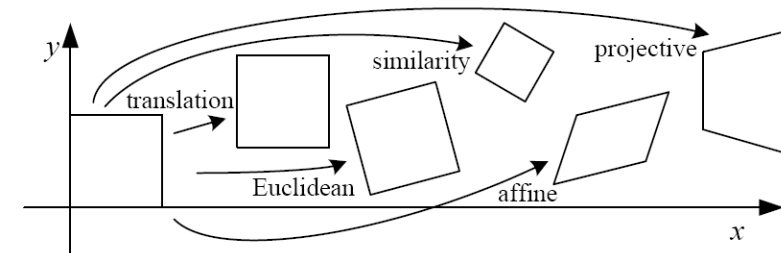


Classification of 2D transformations

Affine:
uniform scaling + shearing
+ rotation + translation

$$\begin{bmatrix} r_1 & r_2 & r_3 \\ r_4 & r_5 & r_6 \\ \underline{0} & \underline{0} & 1 \end{bmatrix}$$

degrees of freedom: 6



Classification of 2D transformations

$$\begin{bmatrix} s \cdot \cos \theta & -s \cdot \sin \theta \\ s \cdot \sin \theta & s \cdot \cos \theta \end{bmatrix} \cdot \begin{bmatrix} 1 & h_1 \\ h_2 & 1 \end{bmatrix} = \begin{bmatrix} s \cdot \cos \theta - h_2 \cdot s \cdot \sin \theta & -s \cdot \sin \theta + h_1 \cdot s \cdot \cos \theta \\ s \cdot \sin \theta + h_2 \cdot s \cdot \cos \theta & s \cdot \cos \theta + h_1 \cdot s \cdot \sin \theta \end{bmatrix} \equiv \begin{bmatrix} r_1 & r_2 \\ r_4 & r_5 \end{bmatrix}$$

similarity (2 DOF)

shear (2 DOF)

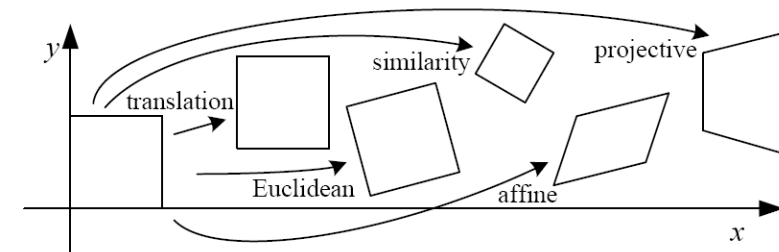
composition (4 DOF)

Degree Of Freedom

2
Affine: 2
uniform scaling + shearing
+ rotation + translation
2

$$\begin{bmatrix} r_1 & r_2 & r_3 \\ r_4 & r_5 & r_6 \\ 0 & 0 & 1 \end{bmatrix}$$

degrees of freedom: 6



Affine transformations

Affine transformations are combinations of

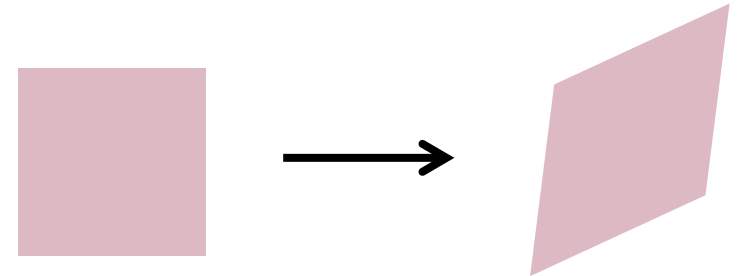
- arbitrary (4 DOF) linear transformations; and
- translations

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

Properties of affine transformations:

- origin does not necessarily map to origin
- lines map to lines
- parallel lines map to parallel lines
- ratios are preserved
- compositions of affine transforms are also affine transforms
- last coordinate does not change

$$w' = w$$



Affine transformations

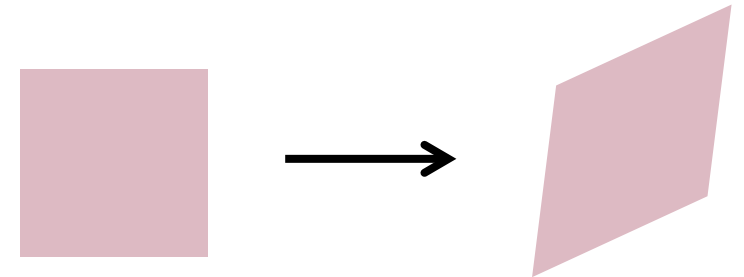
Affine transformations are combinations of

- arbitrary (4 DOF) linear transformations; and
- translations

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

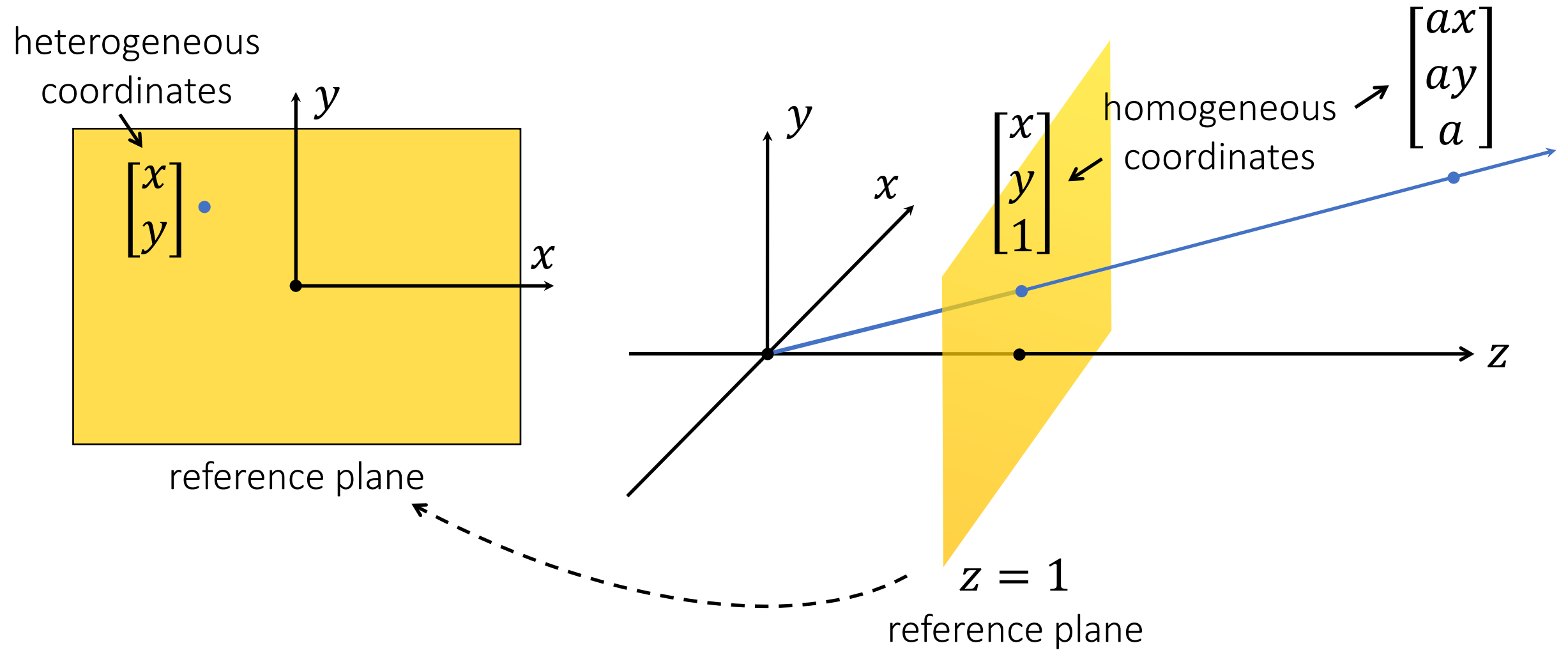
Properties of affine transformations:

- origin does not necessarily map to origin
- lines map to lines
- parallel lines map to parallel lines
- ratios are preserved
- compositions of affine transforms are also affine transforms
- last coordinate does not change

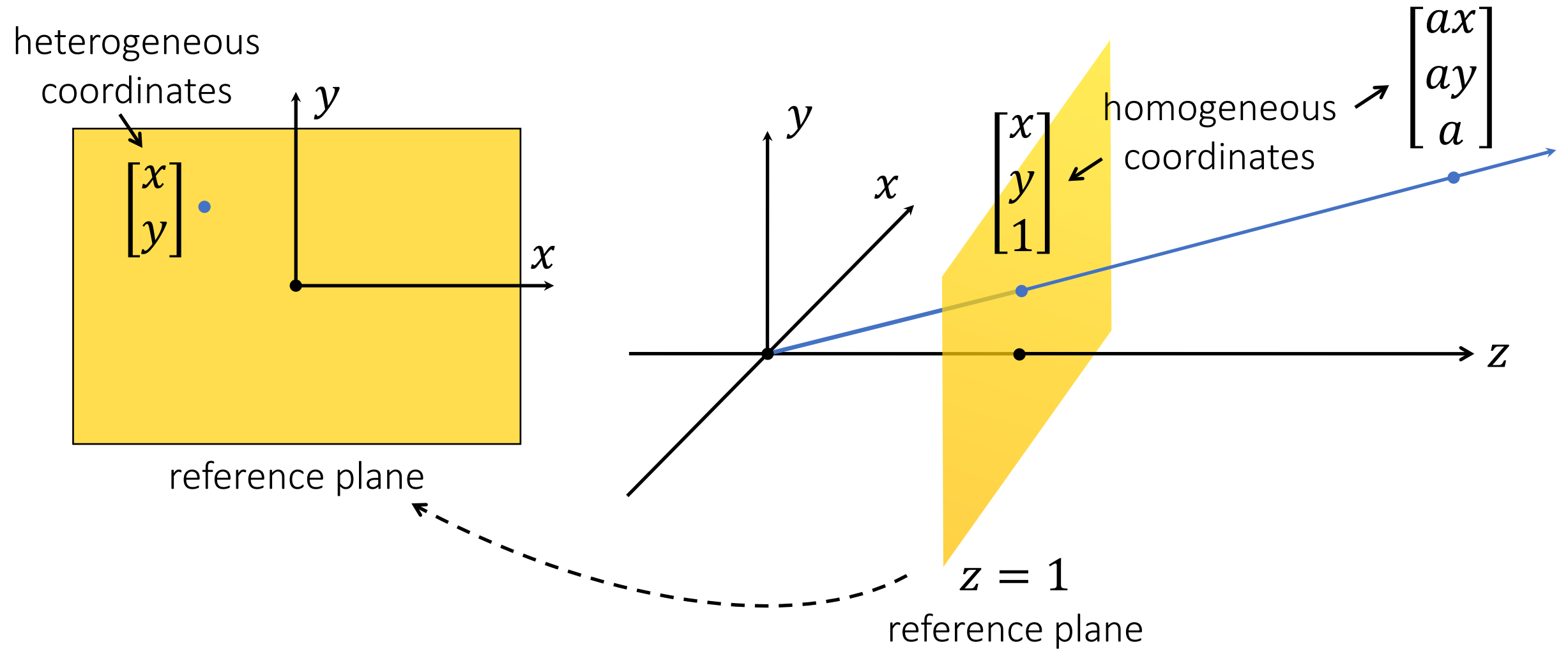


即保持 $Z=C$ 不变，对2D图像在同一平面进行几何变换，

How to interpret affine transformations here?



How to interpret affine transformations here?



By keeping the last coordinate constant, an affine transformation maps points on a $z = c$ plane to points on the same plane.

Classification of 2D transformations

$$\begin{bmatrix} x' \\ y' \\ w \end{bmatrix} = \begin{bmatrix} h1 & h2 & h3 \\ h4 & h5 & h6 \\ h7 & h8 & h9 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

因为是齐次坐标系，方程左右同时除h9

$$\begin{bmatrix} r_1 & r_2 & r_3 \\ r_4 & r_5 & r_6 \\ r_7 & r_8 & r_9 \end{bmatrix} \quad \begin{bmatrix} x1' \\ y1' \\ w' \end{bmatrix} = \begin{bmatrix} \frac{x'}{h9} \\ \frac{y'}{h9} \\ \frac{z'}{h9} \end{bmatrix} = \begin{bmatrix} h1' & h2' & h3' \\ h4' & h5' & h6' \\ h7' & h8' & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Projective (a.k.a. homography):

单应性

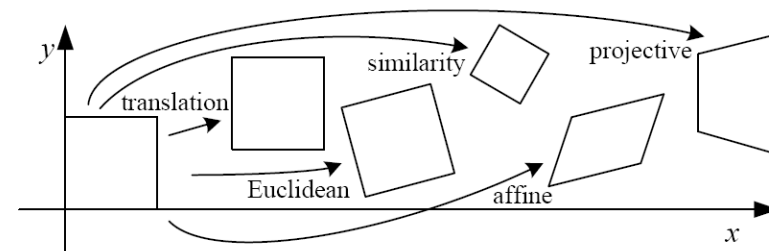
单应性是几何中的一个概念。

单应性是一个从实射影平面到射影平面的可逆变换，
直线在该变换下仍映射为直线。

对应的变换矩阵 就是单应矩阵

单应性变换是对齐次坐标下点的线性变换，
可以通过矩阵运算来表达 $x' = Hx$ $x' = Hx$

degrees of freedom: 8 (homogeneous coordinates, and therefore transformation matrices, are defined up to scale)



Projective transformations (a.k.a homographies)

Projective transformations are combinations of

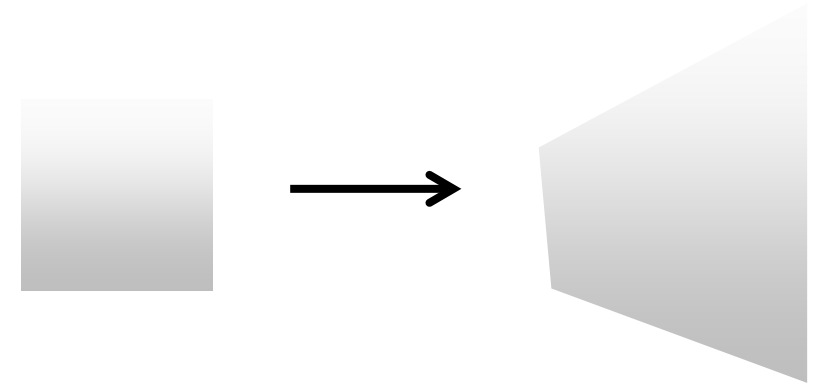
- affine transformations; and
- projective warps

<https://zhuanlan.zhihu.com/p/339949342>

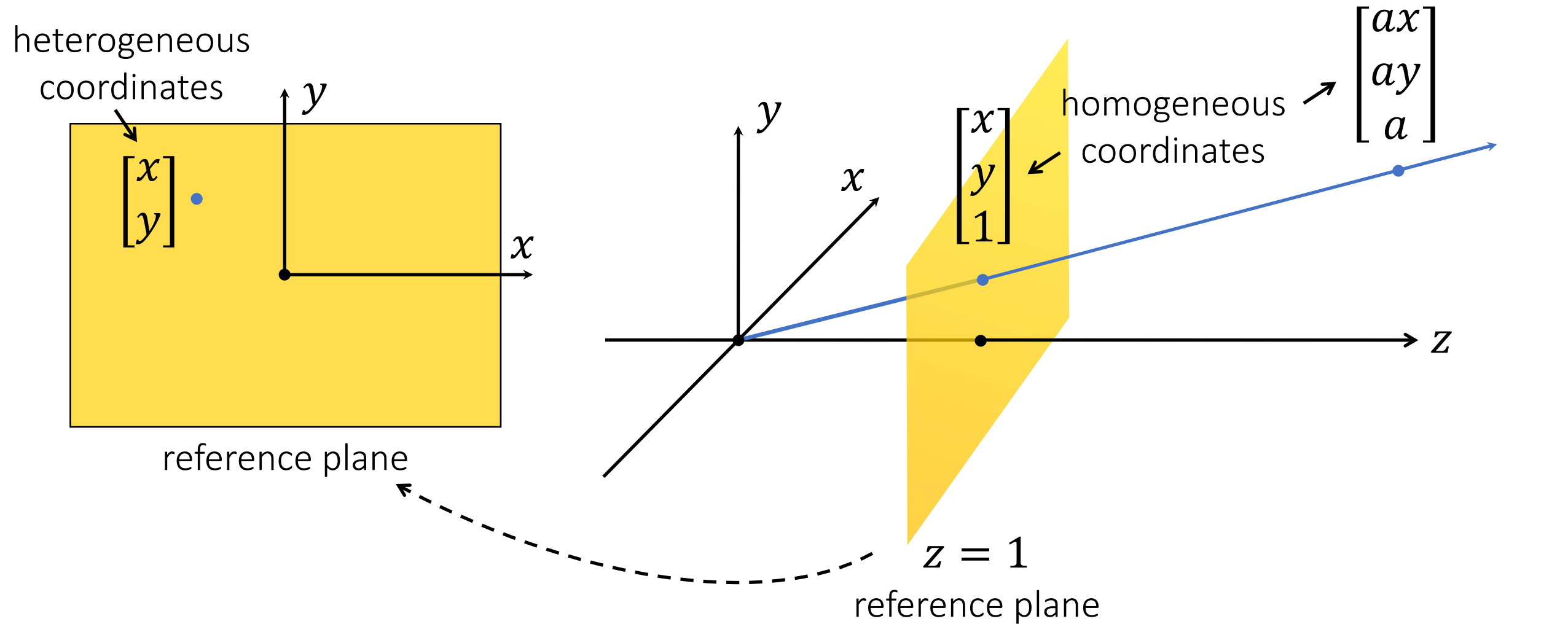
Properties of projective transformations:

- origin does not necessarily map to origin
- lines map to lines
- parallel lines do not necessarily map to parallel lines
- ratios are not necessarily preserved
- compositions of projective transforms are also projective transforms

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$



How to interpret projective transformations here?



By changing the last coordinate, a projective transformation does *not* map points on a $z = c$ plane to points on the same plane.

Classification of 2D transformations

Name	Matrix	# D.O.F.
translation	$\begin{bmatrix} \mathbf{I} & & \mathbf{t} \end{bmatrix}$	2
rigid (Euclidean)	$\begin{bmatrix} \mathbf{R} & & \mathbf{t} \end{bmatrix}$	3
similarity	$\begin{bmatrix} s\mathbf{R} & & \mathbf{t} \end{bmatrix}$	4
affine	$\begin{bmatrix} \mathbf{A} \end{bmatrix}$	6
projective	$\begin{bmatrix} \tilde{\mathbf{H}} \end{bmatrix}$	8